

labadmin / homelab

Home Lab Technical Walkthrough

How the three-node Proxmox lab, its services and its safety gates fit together

Commit	9e4999f95608 (sanitised)
Commit date	2026-10-01
Paper	A4
Links	public

Written for an engineer who has never seen this lab. Every file mention is a link, every code excerpt is pulled from the repository at build time, and the build fails if an excerpt no longer matches its source.

Contents

1	Overview	4
1.1	Hosts	4
1.2	Service catalogue	5
2	Repository map	7
2.1	One source of truth	9
3	Safety model	12
3.1	The approval gate	13
3.2	Destructive tasks	14
3.3	Structural tests	15
3.4	Agent guardrails	16
4	Proxmox cluster	18
4.1	Architecture	18
4.2	Files	18
4.3	Key code	19
4.4	Operate	20
4.5	Recover	20
5	NAS and network	22
5.1	Architecture	22
5.2	Files	22
5.3	Key code	22
5.4	Operate	23
5.5	Recover	23
6	Homelab services (VM 300)	24
6.1	Architecture	24
6.2	Files	24
6.3	Key code	25
6.4	Operate	25
6.5	Recover	25
7	NetBox (VM 330)	26
7.1	Architecture	26
7.2	Files	26
7.3	Key code	26
7.4	Operate and recover	27
8	Media (workstation)	28
8.1	Architecture	28
8.2	Files	28
8.3	Key code	28
8.4	Operate and recover	29
8.5	Music Library	29

9	Observability (VM 310)	30
9.1	Architecture	30
9.2	Files	30
9.3	Key code	30
9.4	Operate	32
9.5	Recover	32
10	Secrets — OpenBao	33
10.1	Architecture	33
10.2	Files	34
10.3	Key code	34
10.4	Operate and recover	36
11	AAA — FreeRADIUS	37
12	Rebuild ladder	38
12.1	Rebuild from zero (L0 → L8)	39
13	CI checks	48
13.1	Pipeline	48
13.2	Local operations	48
13.3	CI checks	48
13.4	Publishing and documentation	49
14	On-demand music library	50
15	Backup and restore	51
15.1	Backup and restore matrix	52
16	Troubleshooting	55
16.1	Lessons learned against the live lab	55
17	Glossary	58
18	Appendix A: Make targets	59
18.1	Make targets	59
19	Appendix B: Workflows	66
19.1	Workflows	66
20	Appendix C: Approval gates	67
20.1	Approval gates	67
21	Appendix D: Operator inputs	70
21.1	Operator inputs (L7)	70
22	Appendix E: Versions and counts	72
22.1	Version matrix	72
22.2	Repository counts	75

1 Overview

This lab is three Proxmox VE 9 nodes in one cluster called `homelab`, a Synology NAS that holds the backups and the private DNS zone, and a Windows desktop (workstation) used only for on-demand workloads. The repository you are reading is the only way changes reach it: a gated, stage-by-stage toolkit of Ansible, OpenTofu and Python. Nothing runs end to end. Every live stage is one Make target that refuses to start until the operator passes an approval flag and an exact confirmation string.

NOTE

Facts in this document come from `docs/reference/environment.md` (not published), `MEMORY.md` (not published) and the source itself, and are true as of the commit printed on the cover. Volatile state (what is running today, open gaps) lives in `MEMORY.md` (not published) and `HOMELAB-ROADMAP.md` (not published); read those before acting.

1.1 Hosts

Host	Address	Alias	Role	Hardware
pve1	192.168.0.11	pve1	Cluster seed; EVE-NG, observability, bao-1	Xeon E-2186M, 12 cores, 31 GiB, 477 GiB NVMe
pve2	192.168.0.12	pve2	Homelab services, bao-2	i5-8250U, 8 cores, 11.5 GiB, 932 GiB HDD
pve3	192.168.0.13	pve3	NetBox, bao-3	i5-6500, 4 cores, 7.6 GiB, 112 GiB SSD
nas1	192.168.0.20	nas	Synology the NAS: backups (NFS), split-horizon DNS, media share	ARM64, 1.6 GiB
workstation	192.168.0.70	workstation	Windows 11 Home: Plex, media guest, music library, staging	Ryzen 7 2700X, 32 GiB, GTX 1080
PiVPN	192.168.0.60	pivpn	WireGuard, IPsec, Pi-hole (unmanaged)	Raspberry Pi

The network is one flat `/22` (`192.168.0.0/24`) behind an the router router. The `192.168.0.x` addresses are inside that same `/22`; they are **not** a separate security zone. The single source of truth for every identity above is `inventory/lab.yml`:

```
14 schema_version: 2
15
16 network:
17   cidr: 192.168.0.0/24           # one flat /22; 192.168.0.x is NOT a separate zone
18   gateway: 192.168.0.1         # the router; never authoritative for the lab zone
19   bridge: vmbr0                # not VLAN-aware on any node
20   dns_zone: lab.example.com
21   lab_resolver: 192.168.0.20    # Synology split-horizon DNS
22   ntp: pool.ntp.org
```

Source: `inventory/lab.yml`, lines 14–22

The network block is what every consumer (OpenTofu through `yamlddecode()`, the Ansible inventories through a contract test) agrees on. The comments carry the two constraints that matter most: the /22 is flat, and the the router must never become authoritative for the lab zone. A `null` anywhere in this file means “not recorded”, and any consumer that needs the value refuses to run.

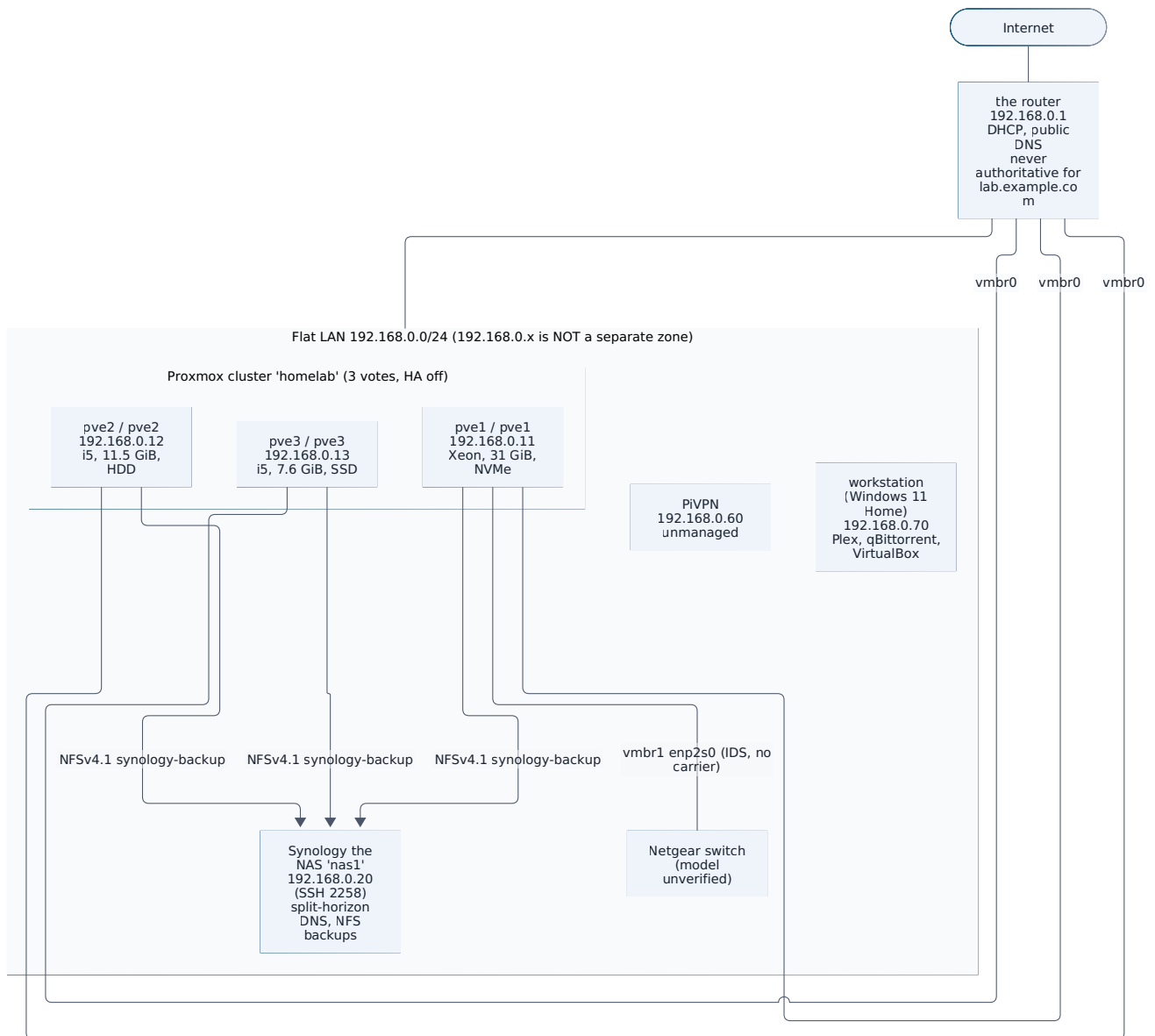


Figure 1. Physical hosts, addresses and the flat /22 network

Diagram source: <docs/walkthrough/diagrams/01-physical-network.mmd>

1.2 Service catalogue

Guest	VMID	Node	Address	What runs there	Component
EVE-NG	290	pve1	—	Network emulation lab (restored from vzdump, never rebuilt)	platform/proxmox/

Guest	VMID	Node	Address	What runs there	Component
homelab-services	300	pve2	192.168.0.30	Nginx Proxy Manager, Homepage, Uptime Kuma, Linkding, IT-Tools, PairDrop, Speedtest Tracker, NetKnife (API + web)	services/homelab-services/
observability	310	pve1	192.168.0.31	Prometheus, Grafana, Loki, Alertmanager, Alloy and five exporters	observability/
bao-1, bao-2, bao-3	320-322	one per node	192.168.0.41-22, VIP .19	OpenBao 2.6.1 Raft cluster, un-initialized	security/secrets-open-bao/
netbox	330	pve3	192.168.0.50	NetBox (six containers)	services/net-box/
media-vm	VirtualBox	workstation	NAT	Radarr, Prowlarr, Recyclarr, exporters, Alloy	services/media/
(native)	—	workstation	—	Plex Media Server, qBittorrent, Music Library	services/media/ , services/music-library/ (not published)

Private sites (`*.lab.example.com`) all terminate TLS at Nginx Proxy Manager on VM 300 with a trusted DNS-01 wildcard certificate, then forward to the guest that serves them.

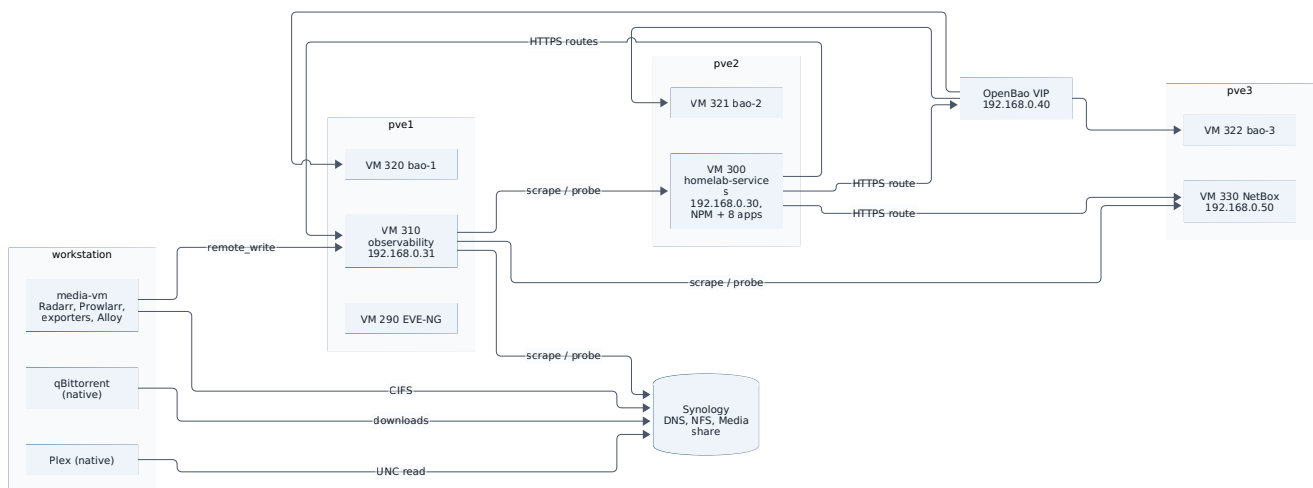


Figure 2. Logical components and how they depend on each other

Diagram source: [docs/walkthrough/diagrams/02-logical-components.mmd](#)

2 Repository map

The monorepo groups components by layer. Each component has its own `Makefile`, `AGENTS.md` and offline checks; the root `Makefile` only dispatches one target to one component (`make COMPONENT=<path> <target>`).

└─ inventory/	lab.yml (single source of truth) + Ansible inventories
└─ platform/	
└─ proxmox/	cluster migration history, guest OpenTofu, rebuild L0–L3
└─ nas/	Synology NFS preflight and registration
└─ services/	
└─ homelab-services/	VM 300 applications
└─ netbox/	VM 330 NetBox
└─ media/	workstation Plex and the *arr guest
└─ music-library/	local music library (desktop + web companion)
└─ observability/	VM 310 stack, dashboards, alerts, backups
└─ security/	
└─ secrets-openbao/	OpenBao cluster, PKI, policies, backups
└─ aaa-freeradius/	FreeRADIUS discovery (no deployment yet)
└─ tests/	cross-component safety contracts
└─ tools/	repo checks, rebuild planner, CI helper, docs, publisher
└─ docs/	decisions, runbooks, reference, walkthrough
└─ .github/workflows/	offline CI, PR plans, dispatch-only deploys

Path	Responsibility	Invoked by	Reads / writes
<code>inventory/lab.yml</code>	Every node, guest, address, MAC, backup job	OpenTofu <code>yamlddecode()</code> , contract tests, rebuild planner	Read only by tooling; edited by hand
<code>inventory/ *.yml</code>	Ansible inventories per component	Each component's <code>ansible.cfg</code>	Read by Ansible
<code>platform/proxmox/</code>	Migration stages 0–11, VM 300/310 adoption, rebuild L0–L3	<code>make -C platform/proxmox <stage></code>	Proxmox hosts over SSH; artifacts/ (git-ignored)
<code>platform/nas/</code>	NFS target preflight and storage registration	<code>make -C platform/nas preflight</code>	Proxmox storage config
<code>services/homelab-services/</code>	VM 300 base, one Compose project per app, app backups	<code>make -C platform/proxmox homelab-service SERVICE=<app></code>	<code>/opt/homelab</code> , <code>/srv/homelab</code> on VM 300
<code>services/netbox/</code>	VM 330 guest, NetBox Compose, <code>pg_dump</code> backups	<code>make -C platform/proxmox netbox-*</code>	VM 330
<code>services/media/</code>	Media guest, *arr containers and settings, Plex checks	<code>make -C services/media <stage></code>	media-vm, NAS media share
<code>observability/</code>	Central monitoring stack and its backup/restore drill	<code>make -C observability <stage></code>	VM 310
<code>security/secrets-openbao/</code>	OpenBao hosts, TLS, HA, post-init policy, Raft backups	<code>make -C security/secrets-openbao <stage></code>	VMs 320–322
<code>tests/</code>	Contracts across components: gates, workflows, denylist, guards	<code>make COMPONENT=tests check, CI</code>	Read only

Path	Responsibility	Invoked by	Reads / writes
tools/	Inventory contract, forbidden constructs, rebuild planner, docs build, publisher	Make targets and CI	Generated docs only
.github/workflows/ (not published)	Offline checks only	GitHub Actions	See CI checks
.agents/ , .claude/ , .co-dex/	Agent skills, guard hook, execution policy	Coding agents	Read only
docs/	Decisions, runbooks, reference, this walkthrough	Humans, make docs	docs/reference/generated/ is generated

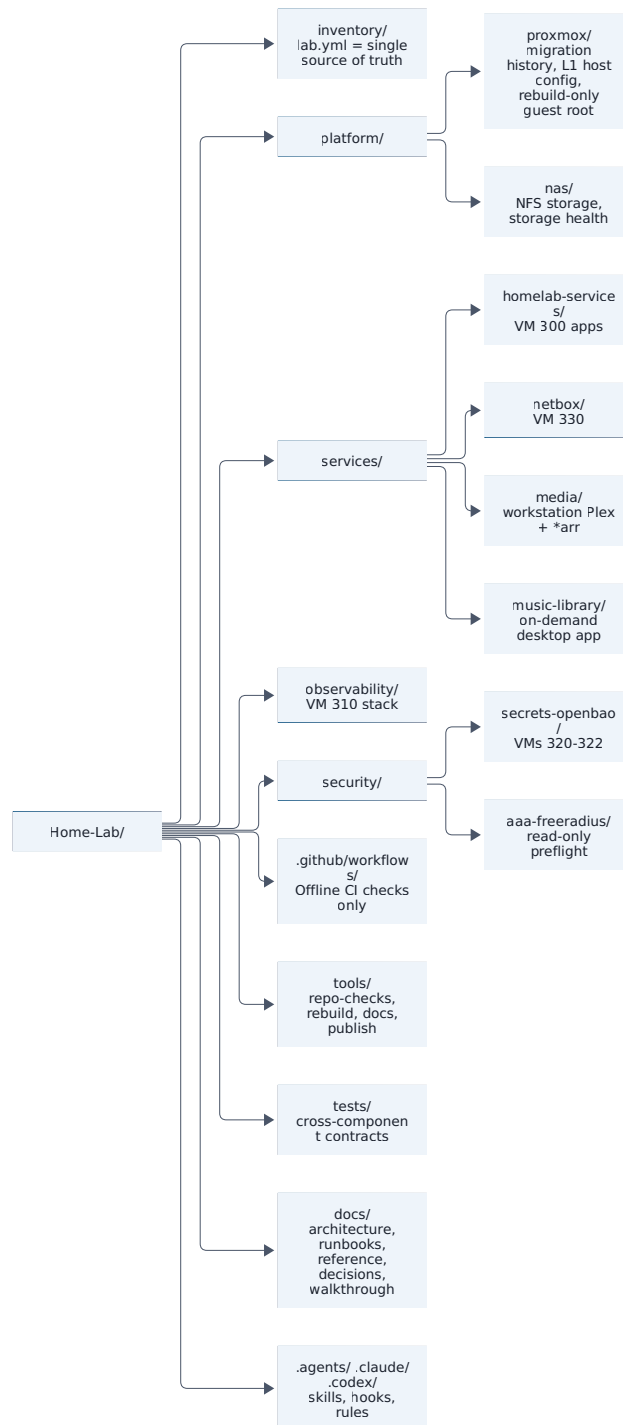


Figure 3. Which directory is responsible for which part of the lab

Diagram source: [docs/walkthrough/diagrams/03-repo-responsibility.mmd](#)

2.1 One source of truth

`inventory/lab.yml` is the single source of truth (ADR-0005). A live identity changes there first and in its consumer in the same commit. `tools/repo-checks/inventory_contract.py` compares the file with every Ansible inventory and OpenTofu root. Deliberate differences are recorded in a ratchet:

```

21 # Drift that exists on purpose, with the reason and the plan to remove it.
22 # A ratchet: an entry that no longer drifts must be deleted (stale_known_drift).
23 KNOWN_DRIFT = {

```

```
24     (
25         "services/netbox/terraform/proxmox-guest/main.tf",
26         "dns_server",
27     ): "Live root still names the NAS's retired .20 address; changing it would make "
28     "a plan against VM 330 propose a cloud-init change. Fix with the state migration "
29     "(ADR-0003), then delete this entry.",
30 }
```

Source: [tools/repo-checks/inventory_contract.py](#), lines 21-30

Each entry is keyed by the consumer file and the key that differs, and its value says why the difference is allowed and how it will be removed. The one current entry is the NetBox root's retired NAS address. Changing it before the state migration would make a plan against the live VM 330 propose a cloud-init change.

```
229 def unexpected(drift) -> list[str]:
230     return [{"w": {k}: {d}} for w, k, d in drift if (w, k) not in KNOWN_DRIFT]
231
232
233 def stale_known_drift(drift) -> list[str]:
234     present = {(w, k) for w, k, _ in drift}
235     return [
236         f"{w}: {k}: no longer drifts; remove it from KNOWN_DRIFT"
237         for (w, k) in KNOWN_DRIFT
238         if (w, k) not in present
239     ]
```

Source: [tools/repo-checks/inventory_contract.py](#), lines 229-239

The ratchet works in both directions. `unexpected` returns drift that is not listed, which fails `test_no_unexpected_drift`. `stale_known_drift` returns entries that no longer drift, which fails `test_known_drift_entries_are_still_real`. An exception therefore cannot outlive its reason.

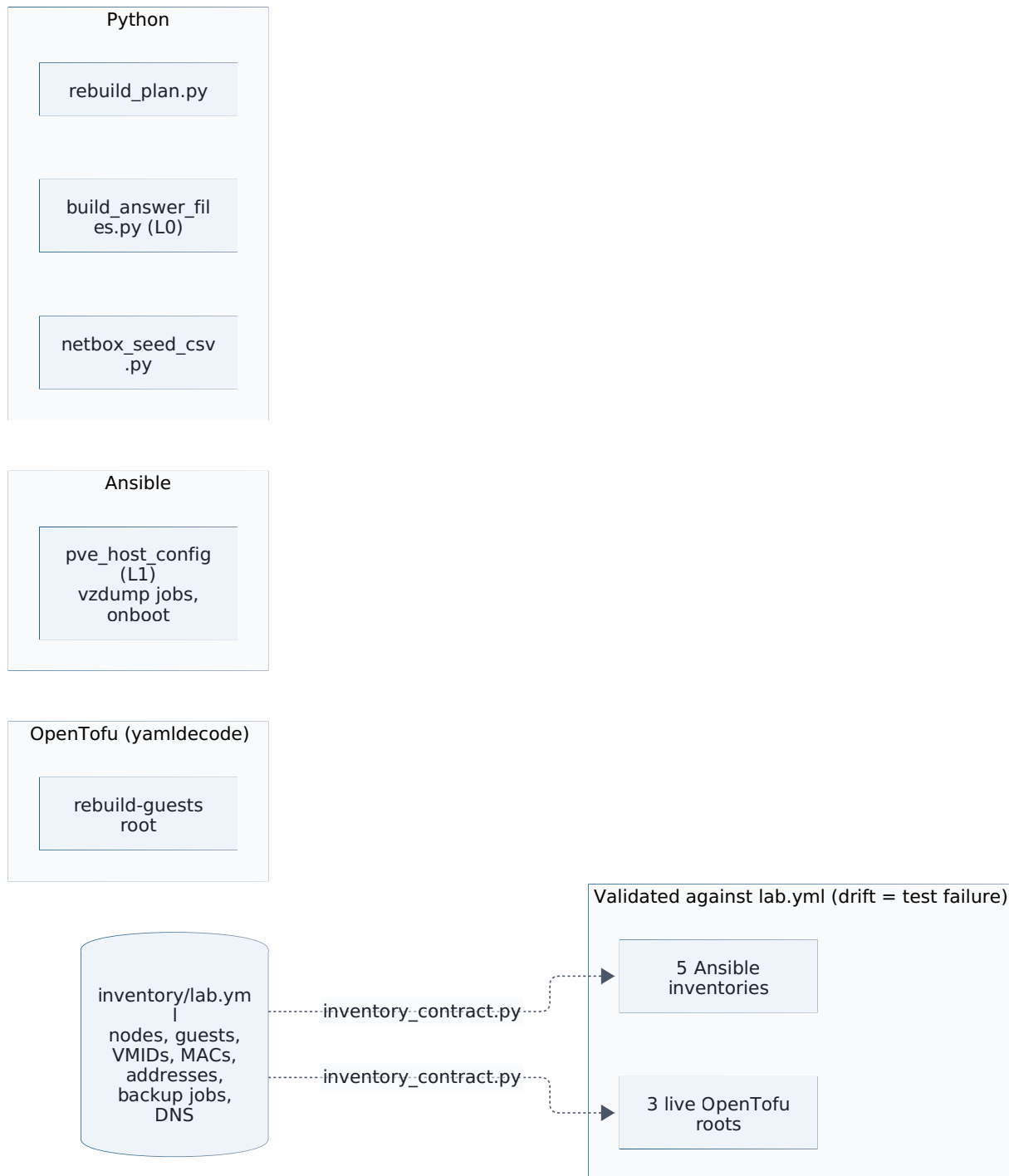


Figure 4. inventory/lab.yaml and the consumers checked against it

Diagram source: <docs/walkthrough/diagrams/04-single-source-of-truth.mmd>

3 Safety model

The lab is live and some of its data is irreplaceable. The repository is built so that the default outcome of any command is “nothing changed”. Every rule below has a mechanism that enforces it and a test that fails the build if the mechanism is removed.

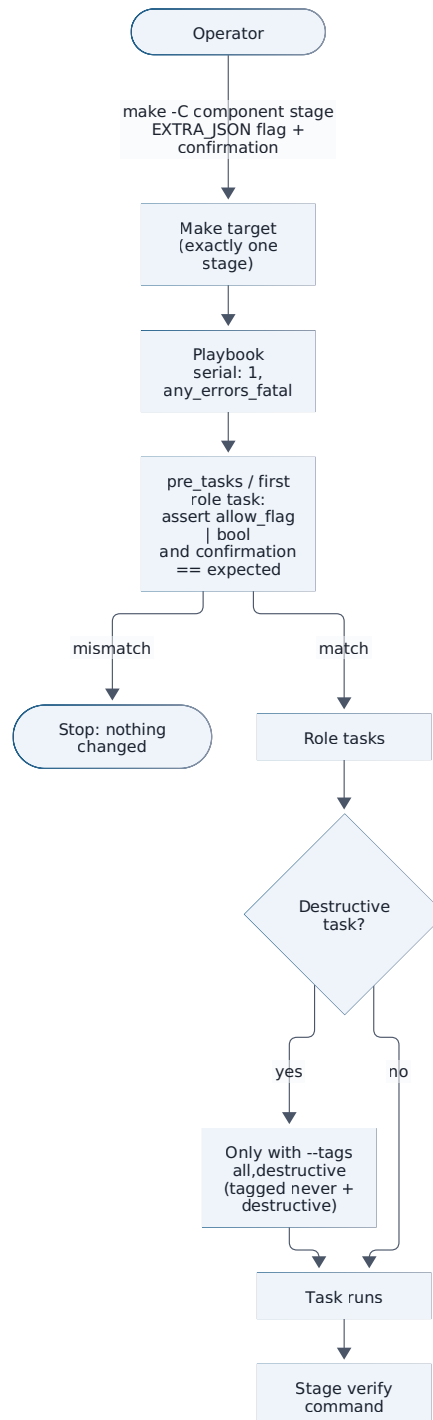


Figure 5. How one gated stage runs, from the operator’s command to the verify step

Diagram source: <docs/walkthrough/diagrams/05-gated-stage.mmd>

Rule	Gate or mechanism	Enforcing test
A live stage needs an approval flag and an exact confirmation	<code>pre_tasks assert</code> in each playbook; flags default <code>false</code> , confirmations <code>""</code>	<code>tests/test_stage_gates.py</code> , <code>test_every_allow_flag_defaults_to_false</code> , <code>test_every_confirmation_defaults_to_empty</code>
Destructive tasks never run by accident	Tagged <code>never</code> + <code>destructive</code> ; need <code>--tags all,destructive</code>	<code>test_destructive_tasks_carry_never_and_destructive</code>
One host at a time, stop on first error	<code>serial: 1</code> , <code>any_errors_fatal: true</code>	<code>test_every_playbook_is_serial_one_and_fatal_on_error</code>
No aggregate target (<code>deploy-all</code> and friends)	Root Makefile only dispatches one target	<code>test_makefile_has_no_aggregate_target</code> , <code>test_no_target_chains_rungs</code>
Backups are never pruned or deleted	NFS storage and every job are <code>keep-all=1</code>	<code>test_no_archive_is_ever_deleted</code> , <code>test_pruning_is_disabled_by_policy</code> , <code>test_no_backup_job_prunes</code>
OpenTofu never destroys a live guest	<code>prevent_destroy</code> , <code>purge_on_destroy = false</code> , import-first adoption; no apply/destroy Make target	<code>test_home_lab_terraform_is_import_first_and_destroy_protected</code> , <code>checks-only workflow contract</code>
SSH host keys are never auto-accepted	<code>StrictHostKeyChecking</code> stays on in every <code>ansible.cfg</code>	<code>test_every_monorepo_ansible_cfg_keeps_host_key_checking_on</code>
No secret is ever tracked	<code>.gitignore</code> , gitleaks in pre-commit and CI, denylist scan	<code>test_no_secret_bearing_path_is_tracked</code> , <code>test_no_secret_shaped_value_is_tracked</code>
No <code>ignore_errors</code> , <code>rm -rf</code> or <code>StrictHostKeyChecking=no</code>	Forbidden-construct scanner with a shrinking baseline	<code>test_tree_has_no_forbidden_construct_beyond_the_baseline</code>
Agents cannot run live commands or read secrets	Claude <code>PreToolUse</code> guard, Codex execution-policy rules	<code>test_forbidden_commands_are_blocked</code> , <code>test_forbidden_commands</code>
GitHub performs hosted checks only	Read permissions, SHA-pinned actions, no secrets or deploy stages	<code>checks-only workflow contract</code>
Every stateful item has a backup or a stated reason	Coverage and backup matrices	<code>test_zero_undocumented_rows</code> , <code>test_every_guest_has_a_backup_job_or_a_reason</code>

3.1 The approval gate

Every live playbook opens the same way. Here is the stage that deploys one application to VM 300:

```

1  ---
2  - name: Deploy exactly one reviewed Home Lab application
3    hosts: homelab_service_guests
4    gather_facts: false
5    become: true
6    serial: 1
7    any_errors_fatal: true
8    # Phase 3: every live stage is gated. Extra vars override these defaults.
9    vars:
10     homelab_allow_service_deploy: false
11     homelab_service_deploy_confirmation: ""
12    pre_tasks:
13     - name: Require this stage's approval and its exact confirmation

```

```

14     ansible.builtin.assert:
15         that:
16             - homelab_allow_service_deploy | bool
17             -
18     homelab_service_deploy_confirmation == ('DEPLOY ' ~ homelab_services_service_name ~ ' ON ' ~
19     inventory_hostname)
20     fail_msg: "Set homelab_allow_service_deploy=true and homelab_service_deploy_confirmation
21     to the exact string in this playbook."
22     quiet: true
23
24 roles:
25     - role: homelab_services

```

Source: [services/homelab-services/ansible/playbooks/40-deploy-service.yml](#), lines 1-22

The play targets one group, one host at a time (`serial: 1`), and aborts every host on the first failure (`any_errors_fatal`). The flag and the confirmation are declared as play variables with safe defaults, so the assert fails unless both are overridden with extra variables. The confirmation embeds the application name and the host, so an approval for one app on one guest cannot be replayed against another. The test below pins this exact shape.

```

51 @pytest.mark.parametrize("playbook", sorted(GATES))
52 def test_stage_is_gated_before_any_role(root, playbook):
53     flag, confirmation, expected = GATES[playbook]
54     (play,) = yaml.safe_load((root / playbook).read_text(encoding="utf-8"))
55     assert play["vars"][flag] is False
56     assert play["vars"][confirmation] == ""
57     first = play["pre_tasks"][0]["ansible.builtin.assert"]["that"]
58     assert first == [f"{flag} | bool", f"{confirmation} == ({expected})"]

```

Source: [tests/test_stage_gates.py](#), lines 51-58

The test loads the playbook as YAML rather than grepping it. It checks that the defaults are `False` and `""` and that the **first** pre-task is the two-line assert, so no role can run before the gate. Confirmations are passed as `EXTRA_JSON`, never as `-e key=value`: Ansible's key/value parser splits multi-word strings (see [lessons learned](#)).

3.2 Destructive tasks

⚠ DANGER — DESTRUCTIVE STAGE

Tasks that stop guests, wipe disks or discard data carry both the `never` and `destructive` tags. They run only when the command line also says `--tags all,destructive`, the stage's `allow_*` flag is `true`, and its exact confirmation string matches. Never add the tags to a command you have not reviewed line by line.

```

16 - name: Require the destructive opt-in tag as well as the approval variables
17     ansible.builtin.assert:
18         that:
19             - "'destructive' in ansible_run_tags or 'never' in ansible_run_tags"
20     fail_msg: |
21         The approval variables are set, but the destructive tasks in this play are
22         tagged `never,destructive` and were therefore not requested.
23
24     Re-run the same command with the explicit opt-in:
25     --tags all,destructive
26     (the Makefile targets for destructive stages already pass this).
27     success_msg: "Destructive opt-in present (--tags all,destructive)."

```

Source: [platform/proxmox/ansible/roles/vzdump_backup/tasks/shutdown_guests.yml](#), lines 16–27

The second assert exists because Ansible silently skips `never`-tagged tasks. Without it, an operator who set the approval variables but forgot the tag would see a green run that shut nothing down and backed up running guests. The assert turns that silent skip into a failure that says exactly what to add.

```

60 - name: Request a graceful ACPI/agent shutdown of every running guest
61   ansible.builtin.command:
62     argv: [qm, shutdown, "{{ item.key }}", --timeout, "{{ guest_shutdown_timeout_seconds }}", --
forceStop, "0"]
63   changed_when: true
64   loop: "{{ vzdump_backup_original_states | dict2items }}"
65   loop_control:
66     label: "{{ item.key }} ({{ item.value }})"
67   when: item.value == 'running'
68   register: vzdump_backup_shutdown_results
69   tags: [never, destructive]

```

Source: [platform/proxmox/ansible/roles/vzdump_backup/tasks/shutdown_guests.yml](#), lines 60–69

The shutdown itself is graceful (`--forceStop 0`), loops only over guests that were recorded as running, and is tagged so it cannot run by default. The recorded power states are written to `artifacts/` `first`, so the rollback knows which guests to start again.

3.3 Structural tests

```

132 def test_every_playbook_is_serial_one_and_fatal_on_error(repo_root):
133     problems = []
134     for path in sorted((repo_root / "ansible/playbooks").rglob("*.yaml")):
135         for play in _plays(path):
136             if not isinstance(play, dict):
137                 continue
138             name = f"{path.name}::{play.get('name', '?')}"
139             if play.get("any_errors_fatal") is not True:
140                 problems.append(f"{name}: any_errors_fatal is not true")
141             targets_hosts = play.get("hosts") not in ("localhost", "127.0.0.1")
142             if targets_hosts and play.get("serial") != 1:
143                 problems.append(f"{name}: serial is not 1")
144         assert not problems, "\n".join(problems)
145
146
147 def test_no_unattended_aggregate_playbook_exists(repo_root):
148     names = {p.name for p in (repo_root / "ansible/playbooks/migration").glob("*.yaml")}
149     for banned in ("site.yaml", "main.yaml", "all.yaml", "40-deploy.yaml", "migrate.yaml"):
150         assert banned not in names
151
152
153 def test_makefile_has_no_aggregate_target(repo_root):
154     text = (repo_root / "Makefile").read_text()
155     for banned in ("deploy-all:", "migrate-all:", "run-all:", "everything:"):
156         assert banned not in text, f"aggregate target {banned} must not exist"

```

Source: [platform/proxmox/tests/test_repo_safety.py](#), lines 132–156

These tests read every playbook and the Makefile. They fail on a play without `any_errors_fatal: true`, a host play without `serial: 1`, a banned playbook name such as `site.yaml`, or an aggregate target such as `deploy-all:`. Local plays against `localhost` are exempt from `serial` because they touch no host.

3.4 Agent guardrails

Coding agents work in this repository. The Claude Code hook [.claude/hooks/guard.py](#) and the Codex rules in [.codex/rules/homelab.rules](#) block live commands and reads of secret-bearing paths:

```

24 PROTECTED_PATH = re.compile(
25     r"""(?ix)
26     (?:\.tfstate(?:\.backup)?$|\.tfstate\.)
27     | (?^\[[/\\])\^[/\\]*\.env$
28     | (?^\[[/\\])\.env\.(?!example$)[/\\]+$
29     | \.tfvars(?:\.json)?$
30     | \.(?:pem|key|p12|pfx)$
31     | (?^\[[/\\])id_(?:rsa|ed25519|ecdsa|dsa)$
32     | _ed25519$
33     | (?^\[[/\\~])\.ssh(?:[/\\]|$)
34     | (?^\[[/\\])artifacts(?:[/\\]|$)
35     | (?^\[[/\\])host-configs(?:[/\\]|$)
36     | (?^\[[/\\])Preferences\.xml$
37     | (?^\[[/\\])config\.xml$
38     | (?^\[[/\\])\.local-credentials(?:[/\\]|$)
39     | (?^\[[/\\])secrets(?:[/\\]|$)
40     | (?^\[[/\\])\.runtime[/\\]radarr(?:[/\\]|$)
41     | (?^\[[/\\])auth\.json$
42     """
43 )

```

OpenTofu/Terraform state
real .env files
.env.<name>, not .env.example
real tfvars
private keys and bundles
SSH private keys
named private keys
~/.ssh/**
machine captures
host configuration captures
Plex token store
*arr API keys
operator secret store
runtime secret directories
quarantined legacy state
agent credentials

Source: [.claude/hooks/guard.py](#), lines 24–43

The guard blocks reading these paths; listing a filename is still allowed. It fails closed: malformed hook input, or any internal error, blocks the tool call. [tests/test_claude_guard.py](#) runs the hook as a subprocess against forbidden commands, wrapped forms (`ssh`, `sudo`, `bash -c`, `xargs`), and ordinary offline work that must stay allowed.

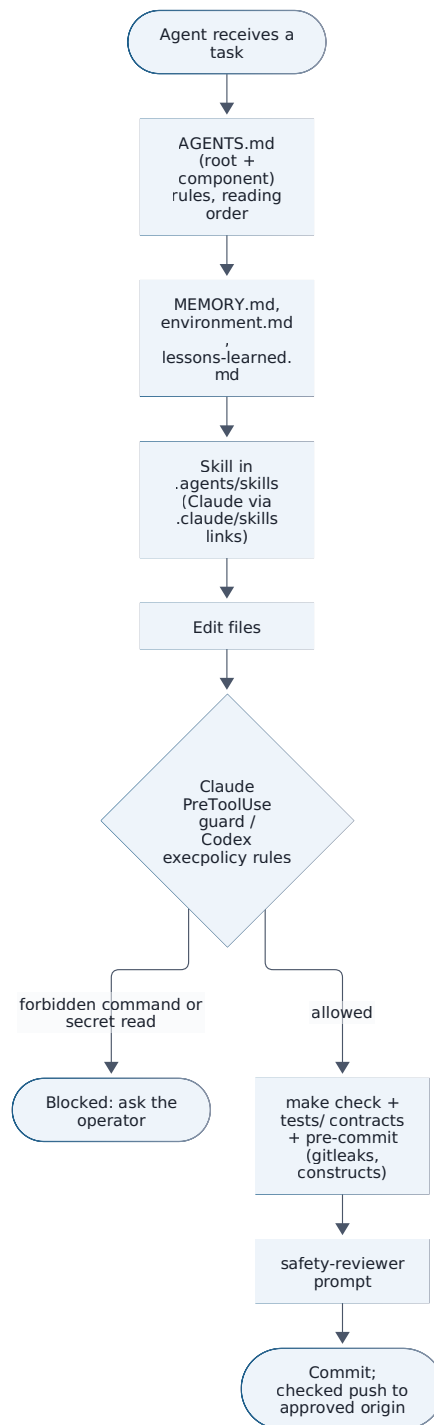


Figure 6. How an agent change moves from task to commit under the guardrails

Diagram source: <docs/walkthrough/diagrams/17-agent-workflow.mmd>

The full list of gates, generated from the playbooks, is in [Appendix C](#).

4 Proxmox cluster

4.1 Architecture

Three standalone PVE hosts were converted into one cluster, `homelab`, without losing a protected guest (stages 0-11, all complete). HA is disabled. There is no shared storage for guests: `pve1` uses `local` (directory on NVMe), and `pve2` and `pve3` each have their own `local-lvm` thin pool, so live migration is not available. Shared storage exists only for backups: `synology-backup`, an NFS export on NAS Volume 1 registered with `--prune-backups keep-all=1`.

OpenTofu owns guest identity (VMID, node, CPU, memory, NIC, protection, startup). It does not own disks, EFI, cloud-init, applications, DNS or passwords. Two kinds of root exist ([ADR-0007](#)): import-first roots that adopt live guests, and one create-mode root that is only ever used to rebuild after a total loss.

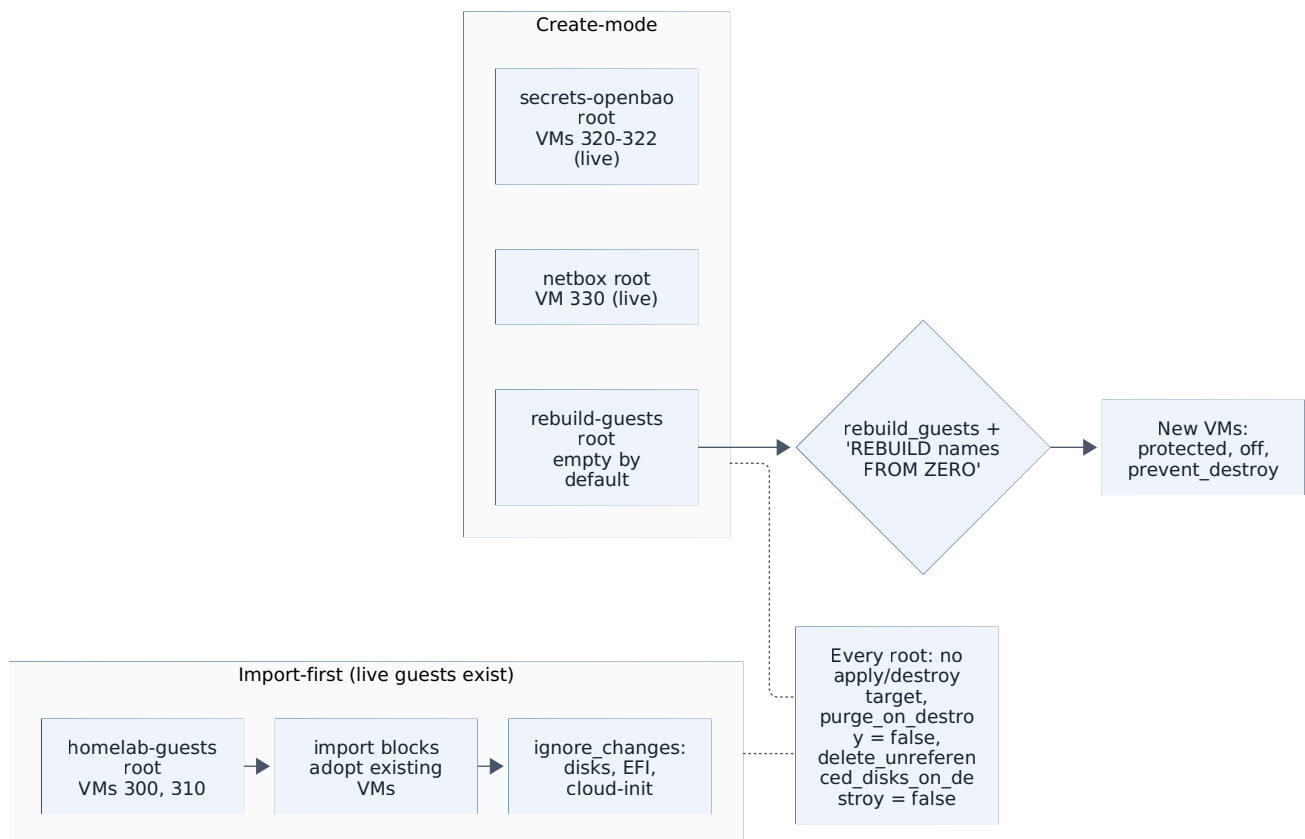


Figure 7. Import-first roots adopt live guests; the rebuild-only root creates them after a loss

Diagram source: `docs/walkthrough/diagrams/07-tofu-import-vs-create.mmd`

4.2 Files

File	Purpose
<code>platform/proxmox/Makefile</code>	Every stage as one target; <code>EXTRA_JSON</code> carries approvals
<code>platform/proxmox/ansible/playbooks/migration/</code>	Numbered migration stages 00-90
<code>platform/proxmox/ansible/roles/</code>	Backup, verify, cluster, join, host-config and guest roles

File	Purpose
platform/proxmox/terraform/homelab-guests/	Import-first adoption of VMs 300 and 310
platform/proxmox/terraform/rebuild-guests/	Create-mode root, driven by <code>lab.yml</code>
platform/proxmox/scripts/	Answer files, restore plans, capacity, VMID collisions, archive verification
platform/proxmox/tests/test_repo_safety.py	The structural safety suite
platform/proxmox/docs/	Migration runbook, rollback, troubleshooting, rebuild catalogue

4.3 Key code

```

1 # Import-first adoption prevents an empty Terraform state from proposing new
2 # VMs over the two live, protected guests.
3 import {
4   to = proxmox_virtual_environment_vm.homelab_services
5   id = "pve2/300"
6 }
7
8 import {
9   to = proxmox_virtual_environment_vm.observability
10  id = "pve1/310"
11 }

```

Source: [platform/proxmox/terraform/homelab-guests/imports.tf](#), lines 1-11

With an empty state, a plain `resource` block would make OpenTofu propose creating VMs 300 and 310 again. The `import` blocks bind the resources to the existing guests by `node/vmid`. The first plan therefore shows an adoption, not a creation.

```

5 resource "proxmox_virtual_environment_vm" "homelab_services" {
6   vm_id      = 300
7   name       = "homelab-services"
8   node_name  = var.homelab_services_node
9   description = local.managed_description
10  tags       = ["homelab-services", "terraform", "ansible"]
11
12  protection          = true
13  on_boot              = true
14  started              = true
15  reboot_after_update = false
16  stop_on_destroy     = false
17  purge_on_destroy    = false
18  delete_unreferenced_disks_on_destroy = false

```

Source: [platform/proxmox/terraform/homelab-guests/main.tf](#), lines 5-18

Proxmox-level `protection` stops the guest being deleted from the UI or API. The three `*_on_destroy` settings make a destroy leave disks alone. `prevent_destroy` in the `lifecycle` block makes OpenTofu refuse any plan that would destroy the resource. The `lifecycle` block also ignores disks and cloud-init, which Ansible and the guest own.

```

9 locals {
10   lab      = yamldecode(file(var.lab_file))
11   rebuildable = { for name, guest in local.lab.guests : name => guest if try(guest.rebuild, false) }
12   selected  = { for name, guest in local.rebuildable : name => guest if contains(var.rebuild_guests, name) }
13   nodes     = toset([for guest in values(local.selected) : guest.node])

```

```

14
15     expected_confirmation = "REBUILD ${join(" ", sort(tolist(var.rebuild_guests)))} FROM ZERO"
16     description           = "Home Lab guest recreated by the rebuild-only root; Ansible configures
    it (L4+)"
17 }
18
19 resource "terraform_data" "rebuild_gate" {
20     count = length(var.rebuild_guests) > 0 ? 1 : 0
21
22     lifecycle {
23         precondition {
24             condition     = length(setsubtract(var.rebuild_guests, keys(local.rebuildable))) == 0
25             error_message = "rebuild_guests may only name guests with rebuild: true in inventory/
lab.yml."
26         }
27         precondition {
28             condition     = var.rebuild_confirmation == local.expected_confirmation
29             error_message = "rebuild_confirmation must equal the exact string \"REBUILD <sorted names>
FROM ZERO\"."
30         }
31     }
32 }

```

Source: [platform/proxmox/terraform/rebuild-guests/main.tf](#), lines 9–32

The rebuild root reads `inventory/lab.yml` directly and considers only guests marked `rebuild: true`. By default `rebuild_guests` is empty, so a plan is empty. To create anything, the operator must list guests by name **and** supply the confirmation `REBUILD <sorted names> FROM ZERO`; both are OpenTofu preconditions.

4.4 Operate

```

make -C platform/proxmox check           # offline: lint, tests, syntax
make -C platform/proxmox validate-ssh    # read-only
make -C platform/proxmox discover        # read-only
make -C platform/proxmox terraform-plan  # import-first plan for 300/310
make -C platform/proxmox rebuild-guests-test # offline, mocked provider

```

There is no `apply` or `destroy` Make target anywhere. Applying a reviewed plan is an operator action, done in the local devcontainer. The operator reviews the plan and supplies the existing local approval; GitHub never plans or applies.

WARNING

`qm set <vmid> --nameserver ...` on a **stopped** cloud-init guest regenerates its SSH host keys on the next boot. Expect a host-key change and verify the new key through `qm guest exec <vmid> -- /usr/bin/ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub` before the operator updates the pin.

4.5 Recover

A node is rebuilt with rungs L0–L2 of the [rebuild ladder](#). The rungs are: an answer-file ISO install, `pve_host_config` for jobs, onboot and startup, and a manual `pvecmd add`. Guests come back from `synology-backup` with `qmrestore` to a spare VMID, all NICs link-down, before anything is swapped in. VM 290 (EVE-NG) is only ever restored from `vzdump`.

DANGER — DESTRUCTIVE STAGE

Rebuilding a node discards its local guests. `pvecmd add`, ISO installs and any restore over a live VMID are manual operator steps. No Make target or workflow performs them.

5 NAS and network

5.1 Architecture

The Synology the NAS (`nas1`, 192.168.0.20) has two jobs here. It is the backup target (NFS on Volume 1), and it is the authoritative DNS server for the private zone `lab.example.com`. Volume 1 is a single 12 TB disk with **no redundancy**, so losing that disk loses every archive at once. Volume 2 is a RAID1 mirror whose Disk 3 has failed SMART and is scheduled for replacement.

D5 approves a WD Red Plus 4 TB CMR replacement in bay 3. The [attended replacement runbook](#) requires a fresh healthy Disk 4 result and a verified independent backup, then DSM Repair and supported post-rebuild checks. RAID1 parity scrubbing is unavailable in DSM; use the actual filesystem's offered scrub operation or record it unsupported and verify SMART/files instead. Volume 1's larger structural risk is addressed separately by adding a 12 TB+ CMR disk in empty bay 1 and converting its Basic pool to RAID1, never Basic to SHR.

DNS is split-horizon **without** the router. The the router must never be authoritative for, or forward, `lab.example.com`: doing so takes the operator's home-automation hub offline. Each guest therefore gets a systemd-resolved routing domain (`Domains=~lab.example.com`, `DNS=192.168.0.20`). Only private-zone queries go to the NAS; everything else stays on the the router, including its IPv6 resolver.

5.2 Files

File	Purpose
platform/nas/ansible/playbooks/05-synology-pre-flight.yml	Read-only NFS reachability and throughput probe
platform/nas/ansible/roles/synology_nfs/	Preflight and gated storage registration
platform/nas/docs/synology-storage-health.md	Disk and pool health, measured options
observability/config/prometheus/file_sd/dns.yml	The only DNS target that may be probed

5.3 Key code

```
8 # `keep-all=1` is what makes this storage a write-only archive from Proxmox's
9 # point of view: no retention policy, so nothing is ever pruned automatically.
10 synology_nfs_prune_policy: "keep-all=1"
```

Source: [platform/nas/ansible/roles/synology_nfs/defaults/main.yml](#), lines 8–10

This one value makes the NFS storage append-only from Proxmox's point of view. With no retention policy, Proxmox never prunes an archive. The same value is repeated on every scheduled job by `pve_host_config`, and a test asserts that no job prunes.

```
1 ---
2 # Authoritative split-horizon DNS check for the private zone.
3 #
4 # ONLY the Synology at 192.168.0.20 is authoritative for lab.example.com.
5 #
6 # Do NOT re-add 192.168.0.1:53 here. The the router is deliberately NOT configured to
7 # serve this zone: making it authoritative takes the home-automation hub offline.
8 # Guests reach the private zone through a systemd-resolved routing domain
9 # (/etc/systemd/resolved.conf.d/lab-dns.conf) that sends only lab.example.com
```

```
10 # to 192.168.0.20, while general resolution stays on the the router. Probing the the router
11 # with the dns_lab module asserts a requirement this architecture rejects and
12 # fires DnsResolutionFailed and EndpointDown permanently.
13 - targets:
14   - 192.168.0.20:53
15   labels:
16     zone: lab.example.com
```

Source: [observability/config/prometheus/file_sd/dns.yml](#), lines 1-16

The comment records a decision that is easy to undo by mistake. Probing the the router for the lab zone would look like thorough monitoring, but it asserts a behaviour the architecture forbids. It would fire `DnsResolutionFailed` permanently.

5.4 Operate

```
make -C platform/nas check          # offline
make -C platform/proxmox nas-preflight # read-only NFS probe from all nodes
ssh nas                            # port 2258, user labadmin; sudo needs a password
```

5.5 Recover

The NAS is rung L0 and is entirely manual. The steps are: restore the DSM configuration backup, recreate the shares (`proxmox-cluster-backups` on Volume 1; `Media` and `media-backups` on Volume 2), recreate the NFS export and the DNS zone, and pin the SSH host key under `HostKeyAlias nas`. DSM has no provider, and its admin credentials are the operator's.

WARNING

Never “simplify” guest DNS by pointing guests wholesale at 192.168.0.20. If the NAS is down, guests would lose all name resolution instead of only the private zone.

6 Homelab services (VM 300)

6.1 Architecture

VM 300 `homelab-services` runs on `pve2` at `192.168.0.30` (Ubuntu 24.04, 4 vCPU, 6 GiB, 120 GiB on `local-lvm`). It runs eight independent Compose projects with nine containers. Every image is digest-pinned and has an explicit memory limit and a health check. Nginx Proxy Manager terminates TLS for every private site, including the observability UIs on VM 310 and NetBox on VM 330. Its admin UI is bound to `127.0.0.1:81` and reached only through an SSH tunnel.

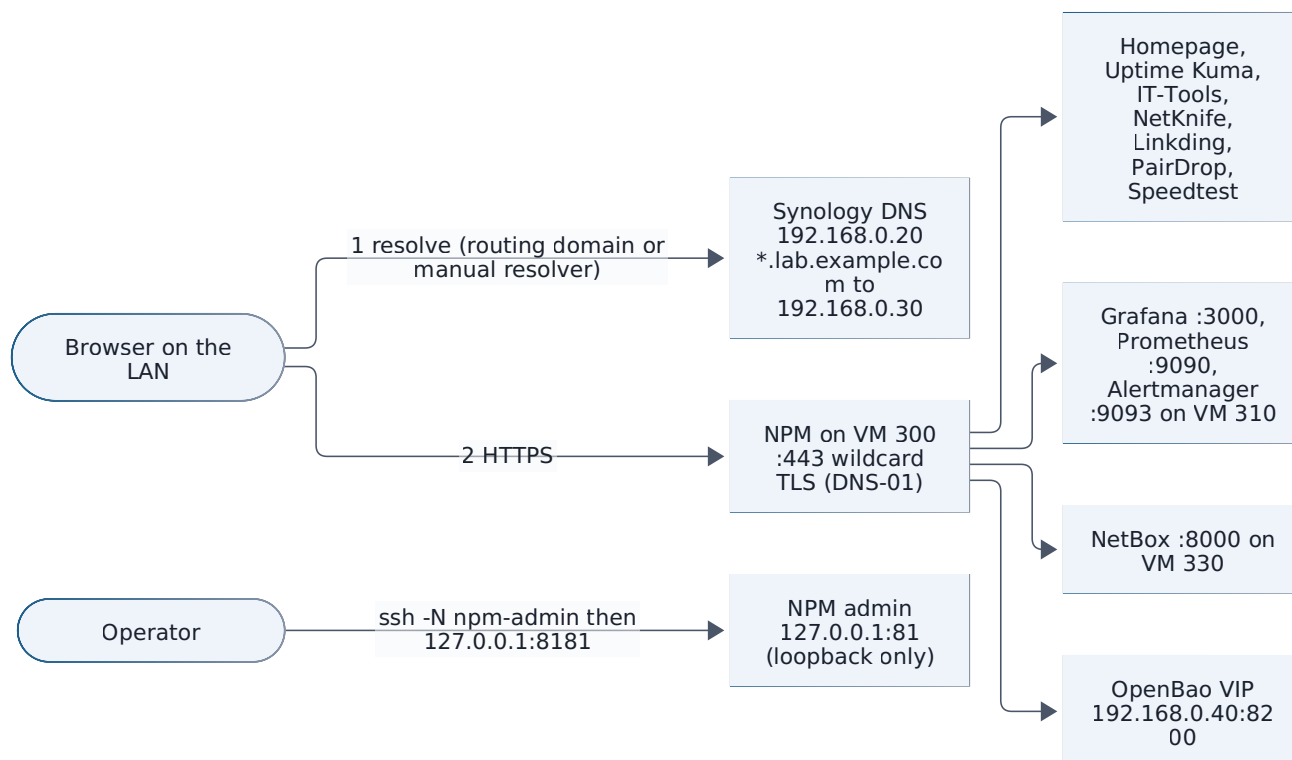


Figure 8. A browser request to a private site: DNS, TLS at NPM, then the backend

Diagram source: <docs/walkthrough/diagrams/08-vm300-request-path.mmd>

6.2 Files

File	Purpose
<code>services/homelab-services/ansible/playbooks/</code>	Guest config (30), preflight (31), one-app deploy (40), backups (50)
<code>services/homelab-services/ansible/roles/homelab_services/tasks/main.yml</code>	Renders exactly one Compose project; NPM route ownership gate
<code>services/homelab-services/config/templates/stage-d/compose/</code>	One <code>compose.yaml</code> per application
<code>services/homelab-services/config/templates/stage-d/images.lock.json</code>	Image digests
<code>services/homelab-services/config/templates/stage-g/backup.py</code>	Application-aware backup

File	Purpose
<code>services/homelab-services/config/uptime-kuma/monitors.yml</code>	Declared monitors, compared with the live dump

6.3 Key code

```

63 def backup_sqlite(source: Path, destination: Path) -> str:
64     if not source.is_file():
65         raise RuntimeError(f"required SQLite database is missing: {source}")
66
67     source_uri = f"file:{source}?mode=ro"
68     with (
69         sqlite3.connect(source_uri, uri=True) as source_db,
70         sqlite3.connect(destination) as destination_db,
71     ):
72         source_db.backup(destination_db)
73         result = destination_db.execute("PRAGMA integrity_check").fetchone()
74
75     if result != ("ok",):
76         raise RuntimeError(f"SQLite integrity check failed for {source}")
77     destination.chmod(0o600)
78     return "ok"

```

Source: `services/homelab-services/config/templates/stage-g/backup.py`, lines 63–78

SQLite files are never copied while the app is running. The source is opened read-only through a URI, and SQLite's online backup API writes a consistent copy. `PRAGMA integrity_check` must return exactly `ok` or the run fails. Uptime Kuma 2 uses MariaDB, not SQLite, so it gets a transactional `mariadb-dump` instead.

6.4 Operate

```

make -C platform/proxmox homelab-preflight          # read-only
make -C platform/proxmox homelab-service SERVICE=homepage \
    EXTRA_JSON='{ "homelab_allow_service_deploy":true,"homelab_service_deploy_confirmation":"DEPLOY
    homepage ON homelab-services"}'
ssh -N npm-admin # then browse http://127.0.0.1:8181 for the NPM admin UI

```

One run reconciles one application. The Homepage card policy is tested: deployed cards need both an `href` and a `siteMonitor`, and planned entries have neither.

6.5 Recover

The backup timer runs at 02:45 and writes to `/srv/homelab-backups/staging`. The 03:15 `vzdump` of VM 300 carries that off the guest. A restore goes to an isolated copy (protected VMID 399, NIC link-down). The SQLite files are copied back with the app stopped, and the Kuma dump is replayed into its MariaDB. This path was proven end to end before VM 300 was set to `onboot=1`.

7 NetBox (VM 330)

7.1 Architecture

VM 330 netbox runs on pve3 at 192.168.0.50 with six containers. UFW admits port 8000 only from VM 300, and NPM serves `netbox.lab.example.com` and `ipam.lab.example.com`. The database starts empty; the rebuild hint from the destroyed pve2 snapshot is not a backup. Its OpenTofu root ([services/netbox/terraform/proxmox-guest/](#)) is separate from the VM 300/310 root on purpose. A drift plan for the adopted guests can therefore never also propose creating VM 330.

7.2 Files

File	Purpose
services/netbox/ansible/playbooks/	Preflight (30), guest config (31), deploy (40), backups (50)
services/netbox/config/templates/stage-a/compose.yaml.j2	NetBox Compose project
services/netbox/config/templates/stage-b/backup.py	pg_dump plus media, hashed, never pruned
services/netbox/scripts/netbox_seed_csv.py	Offline CSV seed built from inventory

7.3 Key code

```

66 def dump_postgres(destination: Path) -> None:
67     """Stream a single-transaction pg_dump straight into a gzip file."""
68     command = [
69         DOCKER,
70         "exec",
71         POSTGRES_CONTAINER,
72         "pg_dump",
73         "--username=netbox",
74         "--dbname=netbox",
75         "--format=plain",
76         "--no-owner",
77         "--no-privileges",
78         "--serializable-deferrable",
79     ]
80     with (
81         subprocess.Popen(command, stdout=subprocess.PIPE, stderr=subprocess.PIPE) as process,
82         gzip.open(destination, "wb", compresslevel=6) as output,
83     ):
84         if process.stdout is None:
85             raise RuntimeError("pg_dump did not open stdout")
86         shutil.copyfileobj(process.stdout, output, length=1024 * 1024)
87         process.stdout.close()
88         returncode = process.wait()
89     if returncode != 0:
90         raise RuntimeError(f"NetBox pg_dump failed with exit code {returncode}")
91     if destination.stat().st_size == 0:
92         raise RuntimeError("NetBox pg_dump produced an empty archive")

```

Source: [services/netbox/config/templates/stage-b/backup.py](#), lines 66–92

The dump runs inside the Postgres container as a single `--serializable-deferrable` transaction. It gets a consistent snapshot without blocking writers. Output streams straight into gzip without touching disk uncompressed. A non-zero exit or an empty file is an error, never a silently truncated backup.

7.4 Operate and recover

```
make -C platform/proxmox netbox-preflight      # read-only
make -C platform/proxmox netbox-deploy EXTRA_JSON='{"netbox_allow_deploy":true,"netbox_deploy_confirmation":"DEPLOY NETBOX ON netbox"}'
```

The backup runs at 03:45 and the 04:45 vzdump carries it off the guest. To restore, load the dump into an isolated VM 330 copy with an empty database, then compare object counts. That isolated restore and `onboot=1` are still gated; see `MEMORY.md` (not published).

8 Media (workstation)

8.1 Architecture

workstation stays Windows 11 Home. Plex Media Server and qBittorrent run natively. Radarr, Prowlarr, Recyclarr, the exportarr exporters and Alloy run in Docker inside `media-vm`, a VirtualBox guest. The library lives on the NAS `Media` share. Sonarr is not deployed. Running Radarr natively on Windows would have needed App Control policy changes, which were rejected. The VirtualBox guest replaced that path.

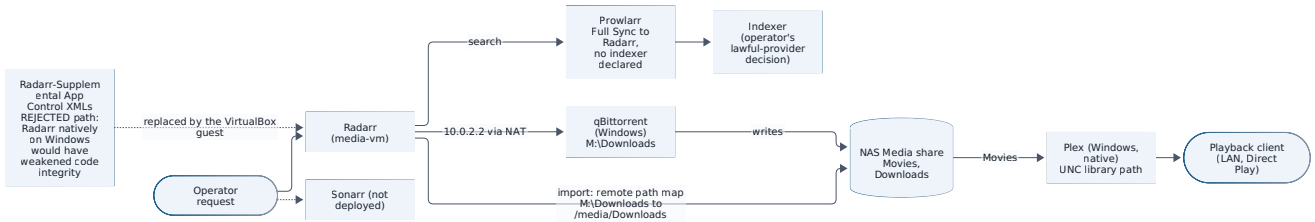


Figure 9. Media request flow: Radarr and Prowlarr in the guest, qBittorrent and Plex on Windows, library on the NAS

Diagram source: `docs/walkthrough/diagrams/09-media-flow.mmd`

8.2 Files

File	Purpose
<code>services/media/config/compose.yaml</code>	Digest-pinned container definitions
<code>services/media/config/cloud-init/</code>	Guest first boot (L0)
<code>services/media/ansible/roles/</code>	<code>media_host</code> , <code>arr_stack</code> , <code>media_backup</code> , <code>plex</code>
<code>services/media/terraform/arr/</code>	*arr settings through the devopsarr providers
<code>services/media/config/recyclarr/</code>	Quality profiles (ADR-0004)
<code>services/media/scripts/plex_libraries.py</code>	Plex library plan (plan by default)

8.3 Key code

```
1  ---
2  name: lux-media
3  # media-vm containers. radarr and prowlarr are the live definitions
4  # recovered from the guest bootstrap (unchanged). Everything else was added in
5  # Phase 3 and is deployed by the arr_stack role under its gate.
6  # API keys are operator-installed env files under /etc/media/secrets (0600
7  # root); Docker reads them as root, so container users never need access.
8  services:
9    radarr:
10     image: lscr.io/linuxserver/
11     radarr:latest@sha256:adb6c09d6b729ea5e642c99cea35af72702ef476bf4763f153299ac5db9f0b4f
12     container_name: radarr
13     restart: unless-stopped
14     environment:
15       PUID: "1000"
16       PGID: "1000"
17       TZ: America/Los_Angeles
18     ports:
```

```
18     - "7878:7878"
19     volumes:
20     - /srv/radarr/config:/config
21     - /mnt/media:/media
22     tmpfs:
23     - /run/radarr-temp:uid=1000,gid=1000,mode=0700
24     cap_drop:
25     - ALL
26     cap_add:
27     - SETGID
28     - SETUID
29     mem_limit: 1g
30     mem_reservation: 256m
31     cpus: 1.0
32     pids_limit: 512
```

Source: `services/media/config/compose.yaml`, lines 1-32

Each image is pinned by digest even when its tag is `latest`, so a rebuild gets the same bytes. Containers drop every capability and add back only `SETUID` and `SETGID`, which the LinuxServer images need to switch users. Each one has hard memory, CPU and PID limits. API keys are never in the file; they come from root-owned env files under `/etc/media/secrets`.

8.4 Operate and recover

```
make -C services/media check           # offline
make -C services/media preflight       # read-only
make -C services/media sync-quality-profiles # Recyclarr --preview unless gated
```

Metrics are pushed from the guest to VM 310 ([ADR-0006](#)). The guest sits behind VirtualBox NAT with loopback-only forwards, so VM 310 cannot scrape it. workstation also sleeps, so alerts treat an absent series as expected. Radarr's and Prowlarr's own scheduled backups are copied nightly, append-only, to the NAS `media-backups` share. Plex's backup task is behind its own gate.

WARNING

Nothing always-on may live on workstation: it sleeps after five hours on AC. Do not stop or reconfigure the Docker Desktop `radarr` container there; that decision is the operator's.

8.5 Music Library

`services/music-library/` (not published) is a local, on-demand music library for workstation. It has a PySide6 desktop app and a FastAPI web companion for other devices on the LAN. It is its own Python project with its own CI job. Its architecture, legal boundaries and operation are documented in `services/music-library/README.md` (not published), which is the single source for that component.

9 Observability (VM 310)

9.1 Architecture

VM 310 runs the central stack on pve1 NVMe: Prometheus (15 days, 8 GiB cap), Grafana, Loki (seven days), Alertmanager (SMTP through a dedicated mailbox), Alloy, and node, cAdvisor, blackbox, SNMP and PVE exporters. That is ten containers, each with a hard memory limit. The Synology is monitored over SNMPv3 read-only and never hosts observability storage. The PVE exporter uses a read-only API token and installs nothing on the hypervisors.

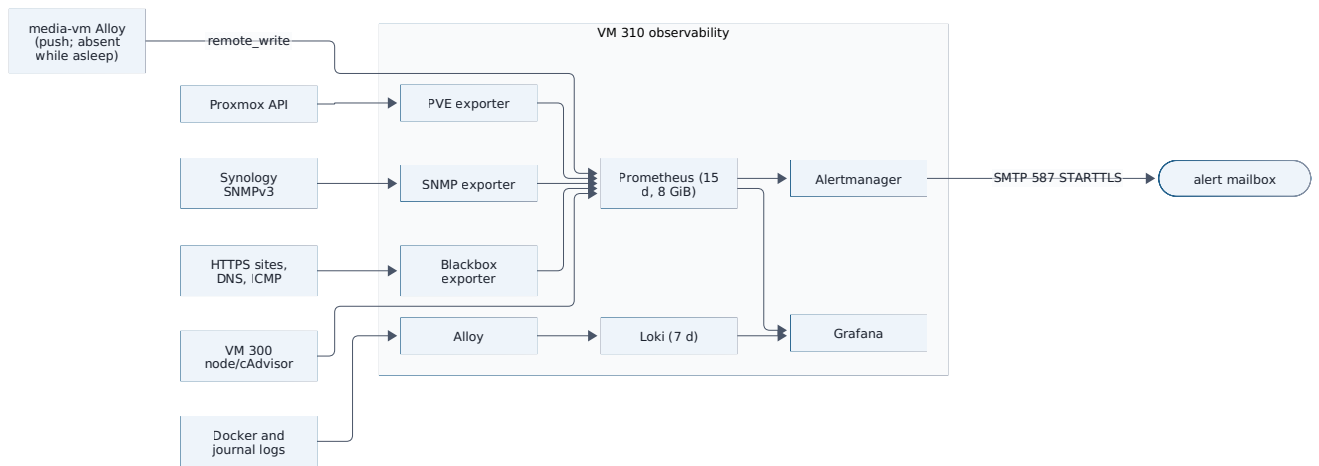


Figure 10. Metrics, logs and alerts: scrape and push paths into VM 310, alerts out by email

Diagram source: [docs/walkthrough/diagrams/10-observability-flow.mmd](#)

9.2 Files

File	Purpose
<code>observability/docker/compose.yaml</code>	The ten-container stack
<code>observability/config/prometheus/</code>	Scrape config, file-based targets, alert rules
<code>observability/config/grafana/</code>	Provisioned dashboards (Synology, Proxmox, fleet, availability)
<code>observability/scripts/backup.sh</code>	TSDB snapshot plus Grafana DB and config archive
<code>observability/scripts/restore-drill.sh</code>	Validates an archive without touching live data
<code>observability/ansible/playbooks/</code>	Bootstrap (20), deploy (40), dashboard update (41)

9.3 Key code

```
12 # Prometheus binds the LAN address, not loopback (see compose.yaml:
13 # ${OBSERVABILITY_IP}:9090:9090). Reading the address from the same .env that
14 # Compose uses keeps the two definitions from drifting. A hardcoded
15 # http://127.0.0.1:9090 here silently failed every nightly run with curl exit 7
16 # and produced no archive at all between 2026-08-24 and 2026-09-06.
17 observability_ip="$(sed -n 's/^OBSERVABILITY_IP=//p' "${stack_root}/docker/.env" 2>/dev/null | tail -1)"
18 readonly prometheus_url="http://${observability_ip:-192.168.0.31}:9090"
```

Source: [observability/scripts/backup.sh](#), lines 12-18

Prometheus binds the LAN address, not loopback. A hardcoded `127.0.0.1` made every nightly backup fail for two weeks with no alert. The script now reads the same `.env` file Compose uses, so the two cannot drift apart again.

```

76 # The script-wide `umask 077` above is correct for the archive, which carries
77 # configuration, but it also made this metric file 0600 root:root. The
78 # node_exporter container runs as `nobody` and could not read it, so the
79 # textfile collector reported node_textfile_scrape_error=1 and
80 # homelab_observability_backup_last_success_timestamp_seconds NEVER EXISTED in
81 # Prometheus. A freshness alert written against it would have been vacuous: it
82 # would evaluate against no series, never fire, and look exactly like health.
83 # Discovered 2026-09-19. The file contains a single Unix timestamp and no
84 # secret, so 0644 is appropriate. Do not "fix" this by relaxing the umask.
85 metric_tmp="${metric_dir}/homelab_observability_backup.prom.tmp"
86 printf 'homelab_observability_backup_last_success_timestamp_seconds %s\n' "$(date +%s)" >"$
{metric_tmp}"
87 chmod 0644 "${metric_tmp}"
88 mv "${metric_tmp}" "${metric_dir}/homelab_observability_backup.prom"

```

Source: [observability/scripts/backup.sh](#), lines 76-88

The freshness metric is written atomically (temporary file, then `mv`), and it is made world-readable on purpose. The comment records why. The first version was `0600`, the exporter runs as `nobody`, and the metric never reached Prometheus. The freshness alert built on it could never fire.

```

51 - alert: ObservabilityBackupMetricMissing
52   expr: absent(homelab_observability_backup_last_success_timestamp_seconds)
53   for: 30m
54   labels:
55     severity: critical
56   annotations:
57     summary: "Backup freshness metric has disappeared"
58     description: >-
59       homelab_observability_backup_last_success_timestamp_seconds is absent
60       from Prometheus, so ObservabilityBackupStale cannot fire and backup
61       failures are invisible. Check the textfile file mode
62       (/var/lib/node_exporter/textfile, must be world-readable), the
63       node_exporter bind mount, and node_textfile_scrape_error.
64
65   # Guard 2: catch the collector failing for any reason - permissions, a
66   # malformed .prom file, or a broken mount - rather than only noticing when
67   # a specific series vanishes.
68 - alert: NodeTextfileCollectorFailing
69   expr: node_textfile_scrape_error > 0
70   for: 15m
71   labels:
72     severity: warning
73   annotations:
74     summary: "node_exporter textfile collector is failing"
75     description: >-
76       node_textfile_scrape_error is non-zero, so at least one .prom file
77       cannot be read. Any alert depending on a textfile metric may be
78       silently inert. Check `docker logs` for the node-exporter container.

```

Source: [observability/config/prometheus/rules/observability.yml](#), lines 51-78

These two rules guard the guard. `absent()` fires if the backup metric disappears.

`node_textfile_scrape_error > 0` fires if any textfile metric cannot be read. Together they stop the staleness alert from silently becoming vacuous again. They must never be deleted.

9.4 Operate

```
make -C observability check      # offline, includes promtool
make -C observability validate   # read-only runtime checks
make -C observability restore-drill ARCHIVE=/srv/observability/backups/<file>
```

9.5 Recover

The backup runs at 03:30 into `/srv/observability/backups` (root-only, so use `sudo -n bash -c '...'` for globbing). The 04:15 `vzdump` carries it off the guest. Loki history is disposable. Rebuilding means: the guest from L3, the stack from the gated deploy, and Prometheus and Grafana state from the latest archive. VM 310 stays `onboot=0` until its restore and onboot gates pass.

10 Secrets — OpenBao

10.1 Architecture

Three OpenBao 2.6.1 voters (VMs 320–322), one on each node, form a Raft cluster. Keepalived holds the VIP 192.168.0.40, where HAProxy terminates HTTP on port 8200 and re-encrypts to each node over TLS. NPM publishes `bao.lab.example.com`. TLS is valid everywhere, and the cluster is deliberately **uninitialized**. Initialization, unsealing and rekeying are attended ceremonies that no automation performs.

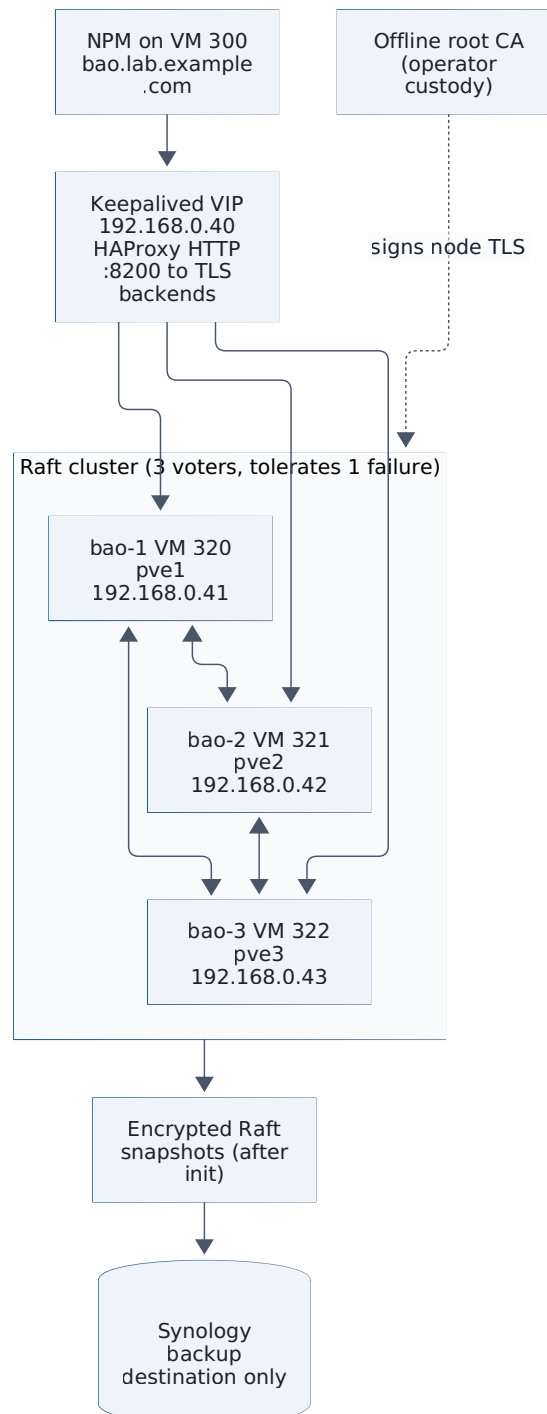


Figure 11. OpenBao voters, the Keepalived VIP and the HAProxy front end

Diagram source: <docs/walkthrough/diagrams/11-openbao-topology.mmd>

10.2 Files

File	Purpose
security/secrets-openbao/ansible/playbooks/	Prepare (10), install (20), TLS (30), HA (40), validate (50), post-init (60/65), backup (70/90), monitoring (80)
security/secrets-openbao/openbao/policies/	Least-privilege policies, applied after init
security/secrets-openbao/scripts/raft_snapshot.py	Encrypted (age) Raft snapshots
security/secrets-openbao/docs/bootstrap.md , unseal.md	The attended initialization and unseal ceremony
security/secrets-openbao/docs/restore-runbook.md	Isolated restore

10.3 Key code

```

90 - name: Query OpenBao initialization state over verified TLS
91   ansible.builtin.uri:
92     url: "https://{{ inventory_hostname }}.{{ openbao_domain }}:8200/v1/sys/init"
93     method: GET
94     ca_path: /etc/openbao/tls/ca.pem
95     return_content: true
96     status_code: 200
97   register: openbao_cluster_init_status
98   changed_when: false
99
100 - name: Stop before initialization and require an attended ceremony
101   ansible.builtin.assert:
102     that:
103       - not (openbao_cluster_init_status.json.initialized | bool)
104     fail_msg: >-
105       This validation gate expects an uninitialized cluster. If initialization
106       was intentional, use the post-init validation stage instead.
107     success_msg: "TLS is valid and OpenBao remains uninitialized. Stop for the attended
    ceremony."
```

Source: [security/secrets-openbao/ansible/roles/openbao_cluster/tasks/main.yml](#), lines 90–107

The validation stage proves two things over verified TLS (the CA path is pinned): every node answers, and `initialized` is still false. If someone initialized the cluster, this stage fails rather than carrying on. The post-init stages are separate playbooks with separate gates.

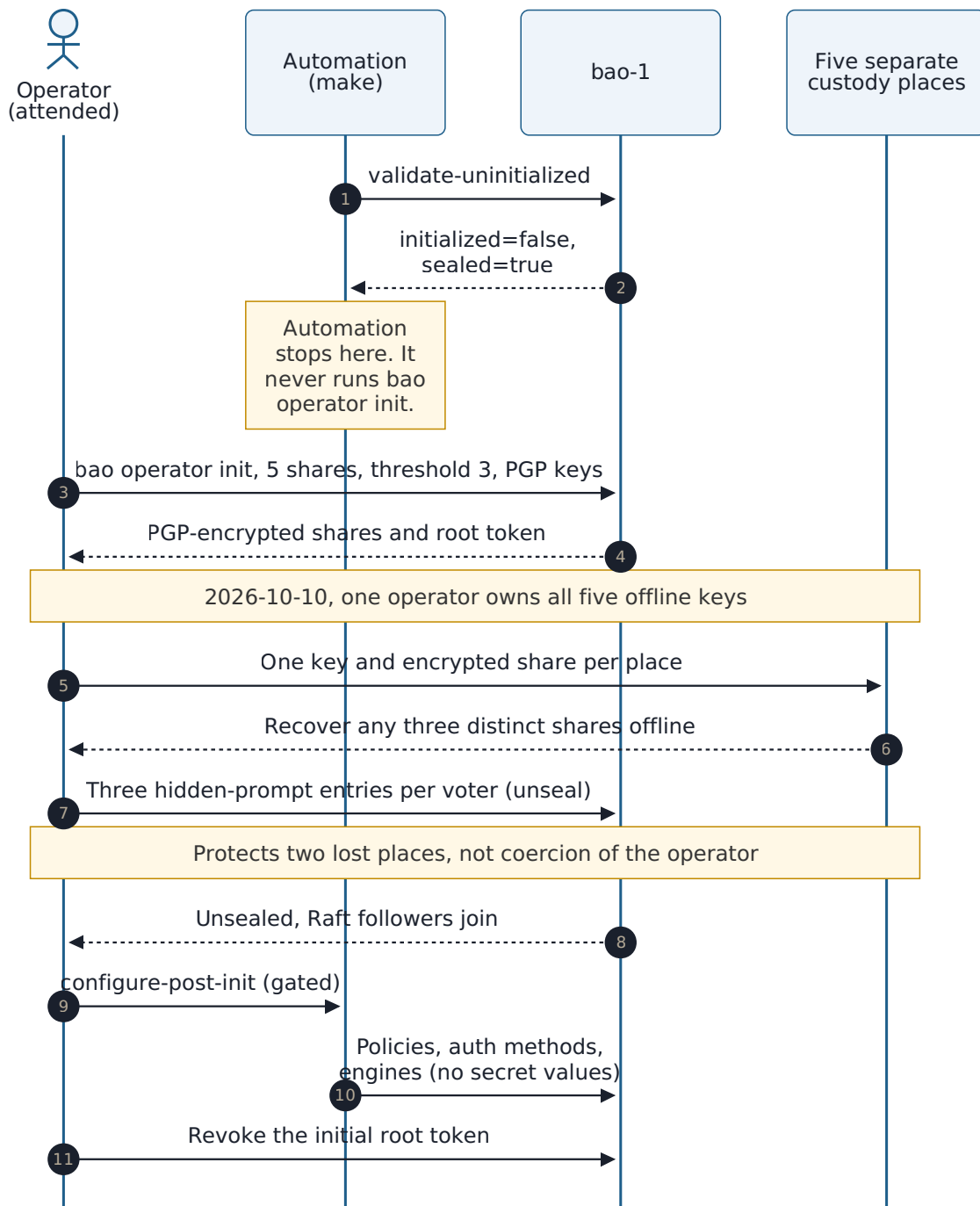


Figure 12. The attended initialization ceremony: automation stops before init

Diagram source: <docs/walkthrough/diagrams/12-openbao-init-ceremony.mmd>

DANGER — DESTRUCTIVE STAGE

Never run `bao operator init`, `unseal` or `rekey` from automation, CI or an agent. The guard hook and the Codex rules block the whole `bao operator` family. D4 schedules the attended ceremony for **2026-10-10**. One operator owns five PGP keys with separate passphrases in five custody places, threshold three. This protects against losing any two places, not coercing that operator. The [ceremony runbook](#) gives offline key generation, fingerprint verification, operator commands and separate post-init gates. The day before requires a verified `vzdump` archive of each VM 320-322. The initial root token is revoked after replacement access works.

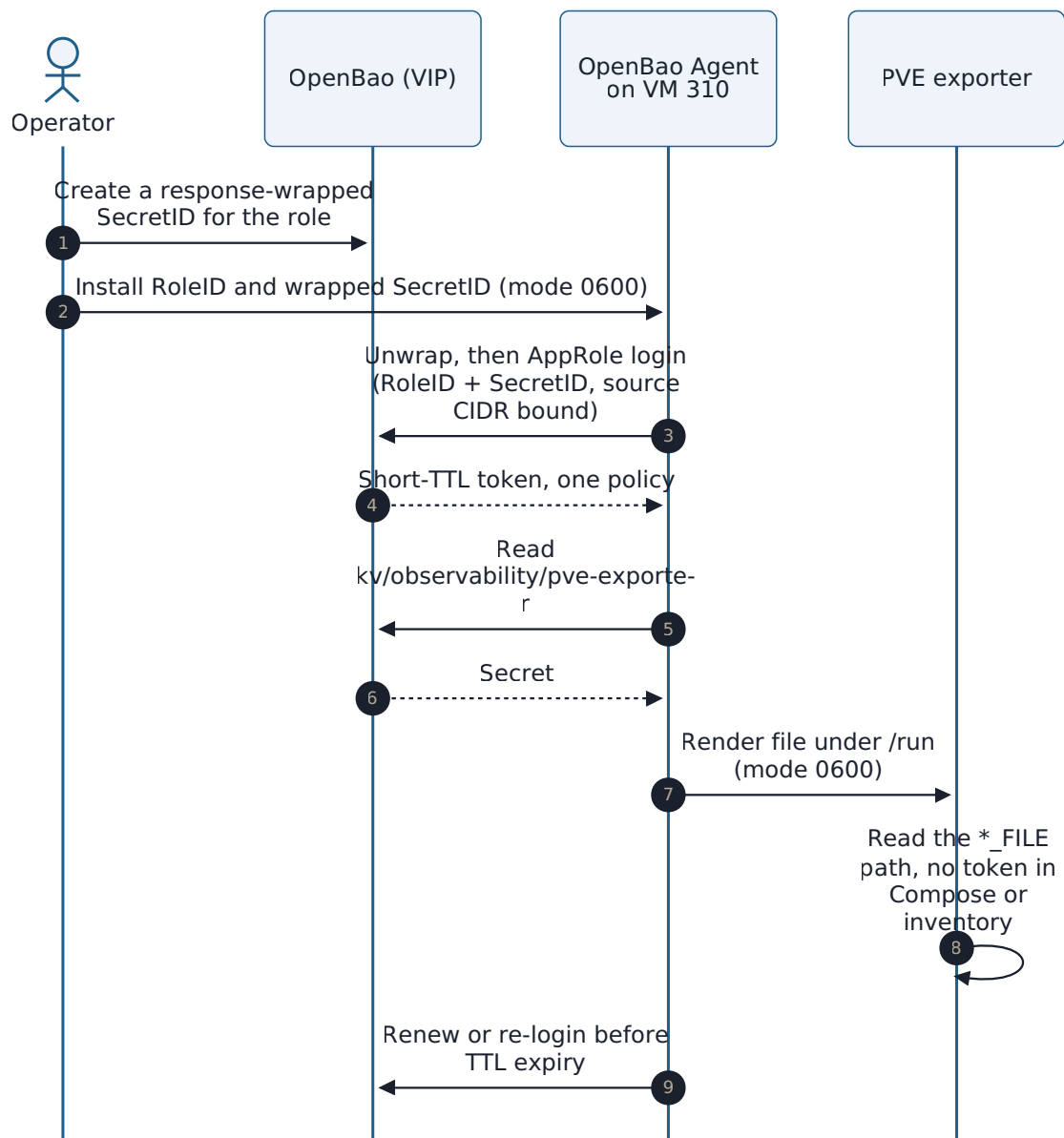


Figure 13. After initialization: an OpenBao Agent renders a secret to a file for the PVE exporter

Diagram source: <docs/walkthrough/diagrams/13-openbao-approle.mmd>

The first planned integration is the read-only PVE exporter token. It uses a response-wrapped AppRole SecretID, an Agent that renders the secret to a `0600` file under `/run`, and a `*_FILE` variable in Compose. No token ever appears in Git, inventory or Compose.

10.4 Operate and recover

```

make -C security/secrets-openbao check # offline
make -C security/secrets-openbao validate-uninitialized # read-only: TLS valid, still uninitialized
make -C security/secrets-openbao recovery-bundle # secret-free configuration bundle
  
```

Before initialization there is no data to lose. The hosts are rebuilt from L3 and the gated prepare, install, TLS and HA stages. After initialization, encrypted Raft snapshots are restored into a protected VM on an isolated bridge by following the restore runbook. The age identity that decrypts them lives outside both OpenBao and the NAS.

11 AAA — FreeRADIUS

[security/aaa-freeradius/](#) holds discovery and a read-only preflight playbook only. Nothing is deployed. The existing `aaa-ldap-radius` VirtualBox guest on the operator's Mac has not been inspected through trusted access. PiVPN has no RADIUS listener. The plan and its required inputs are in [security/aaa-freeradius/docs/aaa-deployment.md](#). RADIUS and LDAP must never be exposed to the Internet.

12 Rebuild ladder

The whole lab can be rebuilt from bare metal by climbing nine rungs, L0 to L8. Each step is one Make target or one manual action, with its own gate, verify command and rollback. The ladder is data ([docs/runbooks/rebuild-ladder.yml](#)), and both the runbook table and `make rebuild-plan` are generated from it:

```
10 rungs:
11   - id: L0
12     title: Bare metal and base installs
13     steps:
14       - id: l0-proxmox-answer-files
15         component: platform/proxmox
16         make: rebuild-l0-answer-files NODE=<node> WRITE=1
17         gate: {kind: local, reason: "Writes git-ignored files only; secrets come from the
operator's environment"}
18         inputs: ["lab.yml nodes.<node>.install.{fqdn,disk_filter,interface_filter}",
PVE_ROOT_PASSWORD_HASH, PVE_ROOT_SSH_KEYS_FILE, PVE_MAILTO]
19         duration: 5 min
20         verify: proxmox-auto-install-assistant validate-answer artifacts/autoinstall/<node>.toml
21         rollback: Discard the generated file; nothing outside the controller changed.
```

Source: [docs/runbooks/rebuild-ladder.yml](#), lines 10–21

A step carries its component, its single Make target, the kind of gate (`local`, `read-only`, `manual`, or a `{flag, confirmation}` pair), the operator inputs it needs, a verify command and a rollback. Tests check that every target exists and that no target chains rungs. They also check that every gate flag and confirmation string really appears in the source.

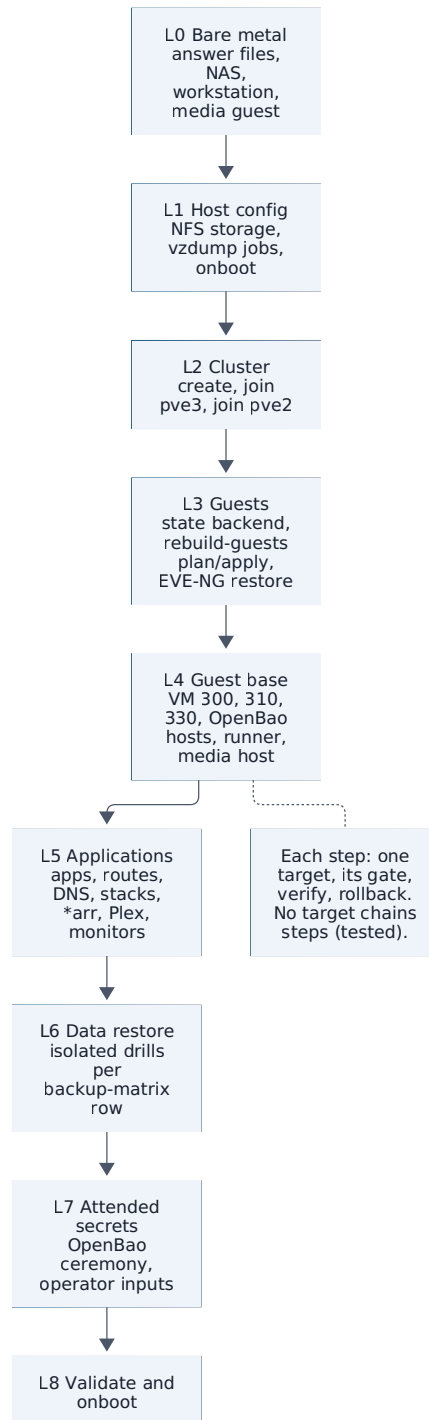


Figure 14. Rungs L0 to L8, from bare metal to validated onboot

Diagram source: <docs/walkthrough/diagrams/06-rebuild-ladder.mmd>

The runbook is included here unchanged. It is the single source for the procedure:

12.1 Rebuild from zero (L0 → L8)

How to rebuild the whole lab from bare metal with this repository plus the backups in <docs/reference/backup-matrix.yml>. Climb one rung at a time:

- Every step is **one** Make target or one manual action. No target chains steps (tested), and there is no aggregate target of any name.

- Run the step's `verify` before the next step. Stop on any surprise.
- Gates are passed as `EXTRA_JSON`, for example `make -C platform/proxmox rebuild-l1-host-config EXTRA_JSON='{"pve_host_config_allow":true,"pve_host_config_confirmation":"L1 CONFIGURE pve pve1 pve2"}'`. An agent prepares these strings (skill `run-gated-stage`); only the operator supplies them.
- `make rebuild-plan` prints this ladder and checks local prerequisites offline (tools, `null` inputs in `inventory/lab.yml`, `backend.hcl`, which environment variables are set — never their values).

Before starting: read [docs/reference/lessons-learned.md](#) (cloud-init regenerates host keys on stopped guests; `/etc/pve` symlinks; the the router must never own `lab.example.com`), [docs/reference/operator-inputs.md](#) (L7) and [docs/reference/coverage-matrix.yml](#) (what is declared, restored or manual).

Known gaps that affect a rebuild are listed in `MEMORY.md`. In particular: live OpenTofu state is still only in the Phase 1 source archive (ADR-0003), and NFS storage registration has a flag-only gate.

12.1.1 The ladder

The table is generated from [docs/runbooks/rebuild-ladder.yml](#) (`python3 tools/rebuild/rebuild_plan.py --markdown`); edit the YAML, not the table. A test fails if they differ.

12.1.1.1 L0 — Bare metal and base installs

Step	Command	Gate	Inputs	Duration	Verify	Rollback
<code>l0-proxmox-answer-files</code>	<code>make -C platform/proxmox rebuild-l0-answer-files NODE=<node> WRITE=1</code>	local: Writes git-ignored files only; secrets come from the operator's environment	<code>lab.yml</code> nodes.. <code>install</code> . { <code>fqdn</code> , <code>disk_filter</code> , <code>interface_filter</code> }; <code>PVE_ROOT_PASSWORD_HASH</code> ; <code>PVE_ROOT_SSH_KEYS_FILE</code> ; <code>PVE_MAILTO</code>	5 min	<code>proxmox-auto-install-assistant validate-answer artifacts/autoinstall/.toml</code> ler changed.	Discard the generated file; nothing outside the control-ler changed.
<code>l0-proxmox-install</code>	manual	manual: ISO preparation and console install are operator steps by design	PVE 9 ISO; answer file from <code>l0-proxmox-answer-files</code>	30 min per node	Compare the console host-key fingerprint with <code>lab.yml</code> nodes.. <code>host_fingerprint</code> , then <code>ssh <alias> pveversion</code>	Reinstall the node; no guest exists yet.
<code>l0-nas</code>	manual	manual: DSM has no provider; NAS admin credentials are the operator's	DSM configuration backup; <code>lab.yml</code> nas	2 h plus RAID re-sync	<code>ssh nas</code> succeeds with the pinned key; shares <code>proxmox-cluster-backups</code> , <code>Media</code> and <code>media-backups</code> exist	Restore the DSM configuration backup.

Step	Command	Gate	Inputs	Duration	Verify	Rollback
l0-workstation	manual	manual: workstation stays Windows and is never wiped by automation	Operator's own Windows backup	operator-dependent	ssh workstation host-name returns workstation	Not applicable; nothing in this repository changes Windows.
l0-media-guest	manual	manual: VirtualBox lifecycle on workstation is attended	services/media/config/cloud-init/user-data.yml.tmpl; admin public key	30 min	make -C services/media preflight	Delete only the new VM; the retained media-vm disks are never touched.

Operator actions:

- `l0-proxmox-install`: `proxmox-auto-install-assistant prepare-iso <pve.iso> -fetch-from iso -answer-file .toml`; boot the node from it; pin the host key after verifying it at the console.
- `l0-nas`: Restore DSM, recreate shares (proxmox-cluster-backups on Volume 1, Media and media-backups on Volume 2), the plex-reader and media-backup accounts, NFS export for the cluster, and the split-horizon DNS zone from lab.yml dns_records.
- `l0-workstation`: Restore Windows from the operator's backup, then install VirtualBox, Plex Media Server and qBittorrent. Plex claim and the client-scoped firewall rule stay attended (L5).
- `l0-media-guest`: Create a 2 vCPU / 4 GiB VM with a 50 GiB data disk, NAT with a loopback-only SSH forward, fill the cloud-init template, build the seed with cloud-localds, attach it and boot.

12.1.1.2 L1 — Host configuration

Step	Command	Gate	Inputs	Duration	Verify	Rollback
l1-nfs-storage	<code>make -C platform/proxmox nas-pre-flight</code>	allow_configure_nfs_storage (flag only) — gap: Flag-only gate predates Phase 3; add a confirmation string	NAS NFS export for the cluster	10 min	<code>pvesh get /nodes//storage/synology-backup/status</code> on each node shows active	<code>pvesm remove synology-backup</code> (removes the storage definition only, never archives).
l1-host-config	<code>make -C platform/proxmox rebuild-l1-host-config</code>	<code>pve_host_config_allow + L1 CONFIGURE pve pve1 pve2</code>	inventory/lab.yml backup_jobs and guest on-boot/startup	5 min	<code>pvesh get /cluster/backup</code> lists every lab.yml backup job	Disable a created job in the UI (never delete archives); <code>qm set -onboot 0</code> .

12.1.1.3 L2 — Cluster

Step	Command	Gate	Inputs	Duration	Verify	Rollback
l2-create-cluster	<code>make -C platform/proxmox create-cluster</code>	<code>al-low_create_cluster + CREATE CLUSTER homelab ON pve1 192.168.0.11</code>	three nodes installed and configured	10 min	pvecm status on pve1 shows one node, quorate	Reinstall pve1 (no guests exist yet).
l2-join-pve3	<code>make -C platform/proxmox join-pve3</code>	<code>al-low_join_pve3 + JOIN pve3 192.168.0.13 TO homelab</code>	root password prompt answered at the console (pvecm add is manual)	15 min	pvecm status shows two nodes, quorate	Reinstall pve3 and rejoin.
l2-join-pve2	<code>make -C platform/proxmox join-pve2</code>	<code>al-low_join_pve2 + JOIN pve2 192.168.0.12 TO homelab</code>	root password prompt answered at the console	15 min	pvecm status shows three nodes, three votes, quorate	Reinstall pve2 and rejoin.

12.1.1.4 L3 — Guests

Step	Command	Gate	Inputs	Duration	Verify	Rollback
l3-state-backend	manual	manual: State migration is the operator's (ADR-0003); agents never migrate state	D:/homelab-state mounted in devcontainer; encrypted NAS state copy or archived plaintext source; per-root TF_VAR_state_passphrase	30 min per root	tofu plan shows no changes after migration	Keep using the archived local state; delete nothing until the plan is clean.
l3-guests-plan	<code>make -C platform/proxmox rebuild-guests-plan REBUILD_VARS=' -var-file=re-build.tfvars '</code>	<code>re-build_guests + REBUILD <sorted guest names> FROM ZERO</code>	backend.hcl (ADR-0003); TF_VAR_proxmox_api_token; TF_VAR_state_passphrase; git-ignored re-build.tfvars with names, confirmation and public keys	10 min	The saved plan creates only the named VMIDs with lab.yml MACs, off and protected.	Discard the plan file.

Step	Command	Gate	Inputs	Duration	Verify	Rollback
l3-guests-apply	manual	manual: There is no apply target; the operator applies the reviewed plan	rebuild.tfplan from l3-guests-plan	20 min	qm list on each node shows the new VMIDs stopped; qm config shows protection: 1	Guests are protected and prevent_destroy is set; removing one is a separate attended decision.
l3-eveng-restore	manual	manual: Restores overwrite a VMID; attended by design	latest verified VM 290 archive on synology-backup	1 h	qm config 290 matches the archive; boot with link_down on every NIC first	Leave the restored VM stopped and protected.

Operator actions:

- `l3-state-backend`: Follow docs/runbooks/state-migration.md in order (OpenBao, NetBox, homelab-guests import-first). Restore encrypted NAS state or re-adopt by import; require a no-change plan before any apply.
- `l3-guests-apply`: `tofu -chdir=platform/proxmox/terraform/rebuild-guests apply rebuild.tfplan`
- `l3-eveng-restore`: `qmrestore 290 -storage local on pve1`, then verify.

12.1.1.5 L4 — Guest base configuration

Step	Command	Gate	Inputs	Duration	Verify	Rollback
l4-vm300-base	<code>make -C platform/proxmox homelab-base</code>	<code>homelab_allow_guest_config + CONFIGURE GUEST homelab-services</code>	VM 300 host key verified out of band through the guest agent	15 min	<code>make -C platform/proxmox homelab-preflight</code>	Re-run with the previous commit's configuration.
l4-vm310-bootstrap	<code>make -C observability bootstrap</code>	<code>observability_allow_bootstrap + BOOTSTRAP observability</code>	VM 310 host key verified out of band	15 min	<code>make -C observability preflight</code>	Re-run with the previous commit's configuration.
l4-vm330-base	<code>make -C platform/proxmox netbox-base</code>	<code>netbox_allow_guest_config + CONFIGURE GUEST netbox</code>	VM 330 host key verified out of band	15 min	<code>make -C platform/proxmox netbox-preflight</code>	Re-run with the previous commit's configuration.
l4-openbao-hosts	<code>make -C security/secrets-openbao prepare-hosts</code>	<code>allow_prepare_openbao_hosts + PREPARE OPENBAO HOSTS 320 321 322</code>	VMs 320-322 host keys verified out of band	20 min	<code>make -C security/secrets-openbao preflight</code>	Re-run with the previous commit's configuration.

Step	Command	Gate	Inputs	Duration	Verify	Rollback
l4-media-host	make -C services/media configure-host	media_host_allow + MEDIA HOST media-vm	/etc/media/secrets/cifs.cred installed root:root 0600	10 min	make -C services/media preflight	Unmount the shares; packages and directories are harmless to leave.

12.1.1.6 L5 — Applications and their configuration

Step	Command	Gate	Inputs	Duration	Verify	Rollback
l5-vm300-apps	make -C platform/proxmox homelab-service SERVICE=<app>	homelab_allow_service_deploy + DEPLOY <app> ON homelab-services	/etc/homelab/secret-store/* for the app; one app per run	10 min per app	Homepage card for the app turns green; uptime_kuma_monitors.py reports no drift	Re-deploy the app from the previous commit.
l5-npm-routes	make -C platform/proxmox homelab-service SERVICE=npm	homelab_services_allow_npm_route_ownership + REPLACE NPM UI PROXY HOSTS WITH DECLARATIVE HOME LAB ROUTES	wildcard certificate DNS-01 credential	15 min	every lab.yml proxied site answers over HTTPS	Restore NPM data from the L6 application backup.
l5-dns-records	manual	manual: Synology DNS has no provider	lab.yml dns_records	20 min	dig @192.168.0.20 .lab.example.com returns 192.168.0.30	Revert the record in DSM.
l5-observability	make -C observability deploy	observability_allow_deploy + DEPLOY OBSERVABILITY STACK ON observability	/etc/observability/secret-store/* (L7 table)	20 min	make -C observability validate	Restore the observability archive (L6).
l5-netbox	make -C platform/proxmox netbox-deploy	netbox_allow_deploy + DEPLOY NETBOX ON netbox	NetBox secrets; on a first deployment also netbox_app_allow_first_deployment with "DEPLOY NETBOX ON VM 330"	20 min	make -C platform/proxmox netbox-preflight	Restore the pg_dump backup (L6).
l5-netbox-seed	manual	manual: Automation never writes to NetBox	lab.yml	15 min	NetBox lists every lab.yml node, guest and address	Delete the imported objects in NetBox.

Step	Command	Gate	Inputs	Duration	Verify	Rollback
l5-openbao-install	make -C security/secrets-openbao install-server	al-low_install_openbao + INSTALL OPENBAO 2.6.1 WITHOUT INITIALIZING	signed Open-Bao 2.6.1 binary	15 min	make -C security/secrets-openbao validate-uninitialized	Stop and disable the service.
l5-openbao-tls	make -C security/secrets-openbao configure-tls	al-low_configure_openbao_tls + DEPLOY OPENBAO TLS TO THREE NODES	operator-issued node certificates from the off-line root	15 min	make -C security/secrets-openbao validate-uninitialized	Re-deploy the previous certificates.
l5-openbao-ha	make -C security/secrets-openbao configure-ha	al-low_configure_openbao_ha + CONFIGURE OPENBAO RAFT AND VIP UNINITIALIZED	VIP 192.168.0.40 reserved	15 min	make -C security/secrets-openbao validate-uninitialized	Stop Keepalived and HAProxy.
l5-media-arr	make -C services/media deploy-arr	arr_stack_allow + MEDIA DEPLOY media-vm	/etc/media/secrets/*.env installed root:root 0600	15 min	Radarr and Prowlarr answer on workstation's loop-back forwards; up{host="media-vm"} appears in Prometheus	docker compose down in /opt/lux-media, then restore *arr backups (L6).
l5-media-arr-config	make -C services/media arr-plan	manual: Apply is manual after the plan is reviewed; no apply target exists	backend.hcl; TF_VAR_radarr_api_key; TF_VAR_prowlarr_api_key; TF_VAR_qbittorrent_*; TF_VAR_state_passphrase	10 min	The plan shows only the root folder, qBittorrent client, remote path mapping and Prowlarr link.	Restore the Radarr/Prowlarr backup zip (L6).
l5-media-profiles	make -C services/media sync-quality-profiles	arr_stack_allow_recyclarr_sync + RECYCLARR SYNC media-vm	/etc/media/secrets/recyclarr.env	5 min	The preview (no gate) shows no changes after a real sync.	Restore the Radarr backup zip (L6).
l5-plex	manual	manual: Claim, client firewall rule and library apply are attended	Plex claim session; PLEX_TOKEN; first client IP	30 min	python services/media/scripts/plex_libraries.py	Remove the library in Plex; NAS media is untouched.

Step	Command	Gate	Inputs	Duration	Verify	Rollback
l5-uptime-kuma	manual	manual: Uptime Kuma 2 has no supported write API	services/homelab-services/config/uptime-kuma/monitors.yml	20 min	python services/homelab-services/scripts/uptime_kuma_monitors.py - dump	Remove the monitor in the UI.

Operator actions:

- `l5-dns-records`: Create the A records in lab.yml dns_records in Synology DNS Server.
- `l5-netbox-seed`: python services/netbox/scripts/netbox_seed_csv.py -out , then bulk import the three files in order.
- `l5-plex`: Claim Plex in the operator's browser, add the client-scoped firewall rule by hand, then run plex_libraries.py plan and, with PLEX_ALLOW_APPLY=true and PLEX_CONFIRMATION="PLEX LIBRARIES workstation", -apply.
- `l5-uptime-kuma`: Create each monitor in monitors.yml in the Uptime Kuma UI.

12.1.1.7 L6 — Data restore (isolated first)

Step	Command	Gate	Inputs	Duration	Verify	Rollback
l6-restore-drills	manual	manual: Restores run only against an isolated VM or network, attended	docs/reference/backup-matrix.yml	1-4 h	Each backup-matrix row's verify command passes on the isolated copy.	Discard the isolated copy; production was never touched.
l6-backup-pull	make -C platform/proxmox configure-backup-pull	allow_backup_pull + INSTALL BACKUP_PULL ON <inventory-node> <address>	dedicated public key; least-loaded mounted node; verified host pins; D free space	10 min plus archive copy	Install the local task; size and SHA-256 match NAS before and after the first pull.	Disable only this task and separately remove only its public key; retain every copy.
l6-media-backups	make -C services/media configure-backup	media_backup_allow + MEDIA BACKUP media-vm	media-backups share mounted	5 min	systemctl list-timers arr-backup.timer on the guest	systemctl disable -now arr-backup.timer (copies are kept).

Operator actions:

- `l6-restore-drills`: Follow every row of docs/reference/backup-matrix.yml, one at a time.
- `l6-backup-pull`: Follow platform/proxmox/docs/backup-pull.md; stop for the exact node confirmation.

12.1.1.8 L7 — Attended secrets and ceremonies

Step	Command	Gate	Inputs	Duration	Verify	Rollback
l7-openbao-init	manual	manual: bao operator init/unseal is an attended ceremony; never automated	five PGP recipients; threshold three	1 h	make -C security/secrets-openbao validate-post-init	None automated; see security/secrets-openbao/docs/restore-runbook.md.
l7-operator-inputs	manual	manual: Secrets are installed by the operator only	docs/reference/operator-inputs.md	1 h	Each stage's secret-file metadata check passes (root:root 0600).	Rotate the affected credential.

Operator actions:

- `l7-openbao-init`: Run the ceremony by hand, then `make -C security/secrets-openbao configure-post-init` with its gate.
- `l7-operator-inputs`: Install or rotate each row of `docs/reference/operator-inputs.md`.

12.1.1.9 L8 — Validate and enable onboot

Step	Command	Gate	Inputs	Duration	Verify	Rollback
l8-validate-cluster	<code>make -C platform/proxmox validate</code>	read-only: Validation only	—	10 min	The report shows every check passing.	Not applicable.
l8-validate-observability	<code>make -C observability validate</code>	read-only: Validation only	—	5 min	<code>make -C observability validate</code> passes with <code>changed=0</code> .	Not applicable.
l8-onboot	<code>make -C platform/proxmox rebuild-l1-host-config</code>	<code>pve_host_config_allow + L1 CONFIGURE</code> <code>pve pve1 pve2</code>	<code>lab.yml</code> onboot set to true only for guests whose L6 drill passed	5 min	A controlled node reboot brings the chosen guests back.	Set onboot back to false in <code>lab.yml</code> and re-run.

Source: [docs/runbooks/rebuild-from-zero.md](#)

13 CI checks

13.1 Pipeline

GitHub runs offline checks on hosted runners with no lab credentials. Plans and deployments run locally from the devcontainer, one gated stage at a time; there is no GitHub runner inside the lab (ADR-0009).

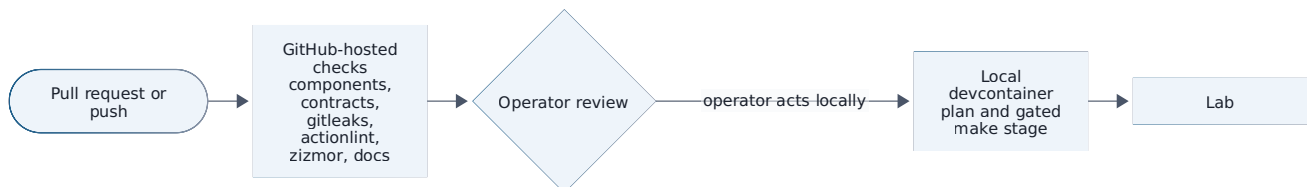


Figure 15. GitHub checks end at review; the operator runs gated local stages

Diagram source: [docs/walkthrough/diagrams/15-cicd-pipeline.mmd](#)

Excerpt from `.github/workflows/ci.yml`, lines 1-15: not published in this copy.

`permissions: {}` at the top means every job must ask for exactly the scopes it uses. The `on:` key is quoted so YAML 1.1 parsers do not read it as `true`. Every action is pinned by full commit SHA, which a contract test enforces.

13.2 Local operations

The operator reviews the change, then runs its existing local Make target with the exact gate. GitHub supplies neither approvals nor credentials.

13.3 CI checks

GitHub checks source on hosted runners; it has no path into the lab. See [ADR-0009](#).

13.3.1 Checks and permissions

`ci.yml` selects changed components and calls `_component-checks.yml`. Contracts, gitleaks and docs always run. Music Library keeps its own lint, test and import job. Actions are pinned by full SHA; workflow permissions default to empty and jobs request read access only.

Actionlint checks syntax. Zizmor is installed at a pinned version with a pinned SHA-256 and runs offline audits. There are no configured GitHub secrets, environments, lab runners, deploy dispatches or plan jobs.

13.3.2 Repository setup

Private origin: [HomeLab-Private](#). Public origin: [HomeLab-Public](#). Enable Actions on the private repository using hosted runners only. Do not register a self-hosted runner or configure lab credentials. Renovate may open dependency PRs; auto-merge stays disabled. Public workflows remain withheld.

13.3.3 Local operations and rollback

After checks pass, review the change and run one gated local Make target in the devcontainer. Approval flags and exact confirmations are the operator's. State plans and migrations follow ADR-0003 and its migration runbook. No GitHub event can authorize or launch a lab stage.

A failing check stops publication or pushing until repaired. Revert a faulty source change with a normal commit; never force-push. CI does not change production. Deployment rollback follows the component runbook.

Source: [docs/runbooks/ci-cd.md](https://docs.runbooks/ci-cd.md)

13.4 Publishing and documentation

A sanitised public copy is produced by `tools/publish/` (not published) (Phase 6). Its origin is the approved public repository. This document is built by `tools/docs-build/build.py` from Markdown, Mermaid sources and the generated reference. The build fails on any warning, broken link or anchor, or changed snippet source.

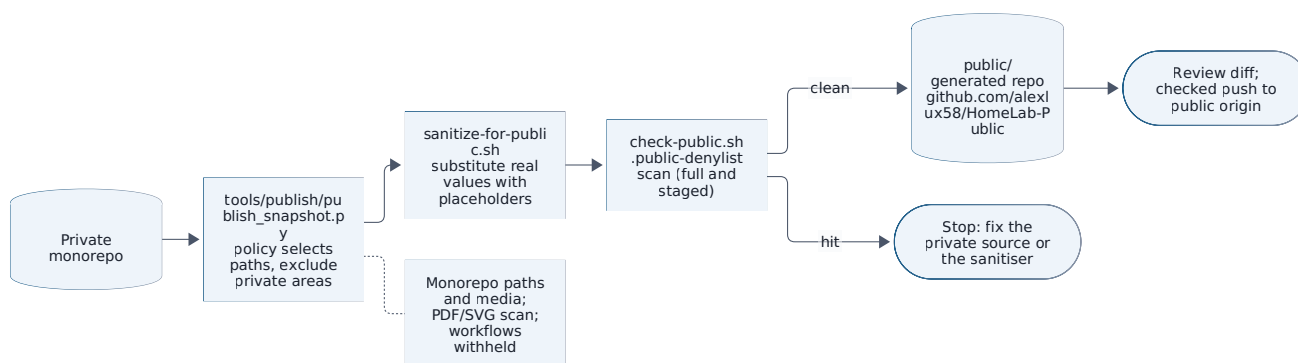


Figure 16. From private source to the sanitised public copy and this PDF

Diagram source: [docs/walkthrough/diagrams/16-publish-pipeline.mmd](https://docs.walkthrough/diagrams/16-publish-pipeline.mmd)

14 On-demand music library

The private Music Library app (not published) runs on demand on workstation. Completed downloads are checksum-verified into the user's Music folder and NAS Media/Music. Tempo is estimated locally and used for ten-BPM range folders; unavailable analysis uses Unknown BPM. A sortable BPM column and editable metadata expose the result. Tempo estimation can report half/double tempo, so the operator can correct it.

The prior packaged app saved downloads inside its bundle instead of the user Music folder. Three existing tracks now have verified copies in both destinations. The desktop launcher targets the new BPM bundle; close the old app, restart, and use Organize by BPM once to redirect its existing catalog to those copies. Existing song originals and the only live catalog remain preserved. This app's source is withheld from the public reading copy.

15 Backup and restore

Every stateful item has a source, a destination, a schedule, a restore method and a verify step. Restores always go to an isolated guest first: a spare protected VMID with every NIC link-down. Nothing prunes. Destinations are append-only, and Proxmox storage is keep-all=1.

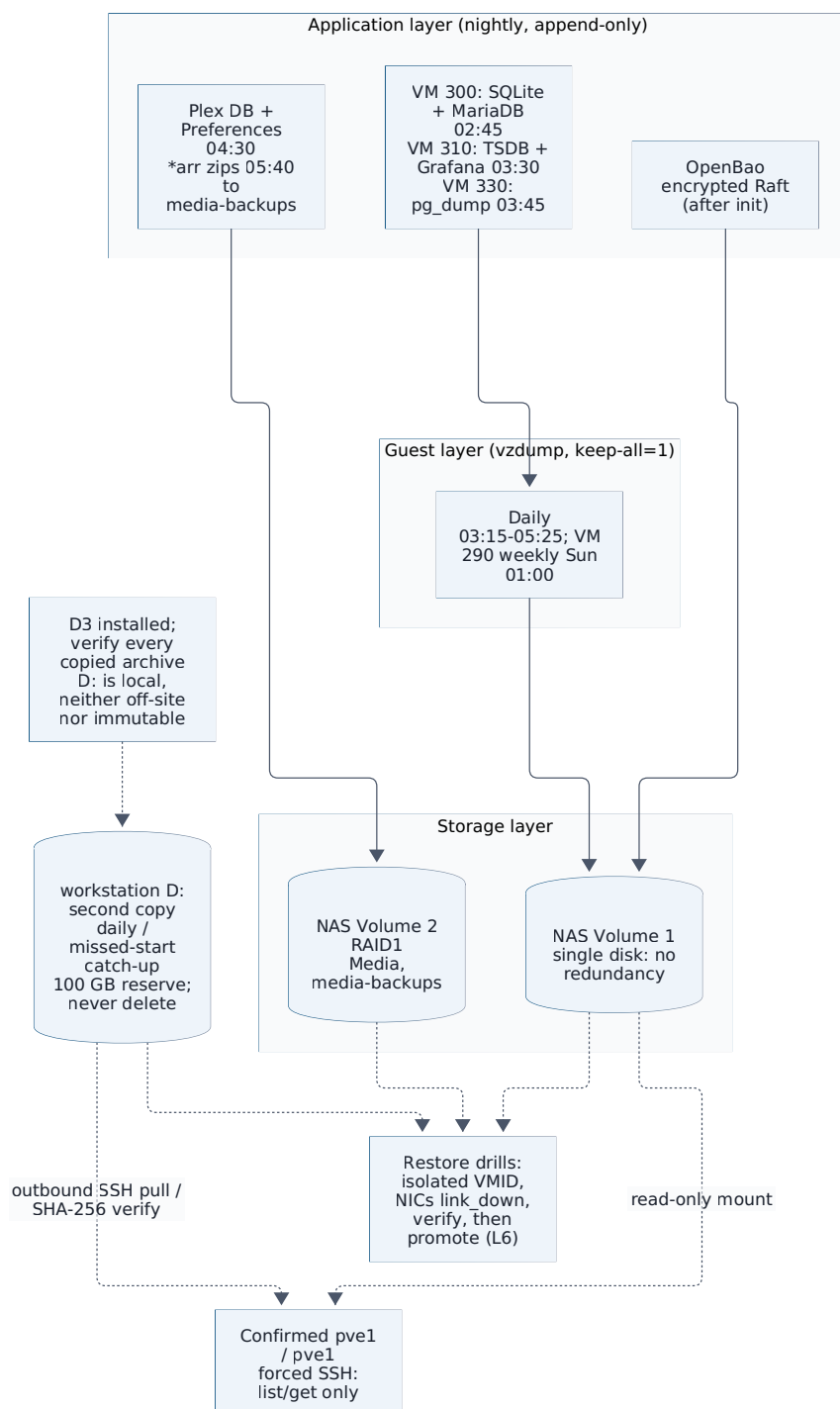


Figure 17. NAS backups, the approved read-only D: pull copy, and isolated restore paths

Diagram source: <docs/walkthrough/diagrams/14-backup-restore.mmd>

The **D3 pull stage** restricts one key to list/get on the NAS-mounted archive directory of one confirmed node. workstation initiates SSH and verifies hashes before publishing each D: copy; its daily task catches missed starts while awake, never wakes the PC and stops below a 100 GB reserve. The exact pve1 installation confirmation was supplied on 2026-09-30. The public key and all three host pins were verified; the gated preflight confirmed pve1 as the least-loaded mounted node. The operator ran that one stage with an interactive password prompt, installing the server and restricted key. The Windows task is installed. Accept each D: copy only after its SHA-256 matches the NAS before and after the pull; registration or a running task alone does not prove that acceptance. Copies are never pruned. D: is a second on-site copy exposed to desktop ransomware. Future off-site data transfer must be an outbound push; GitHub has no path into the lab (ADR-0009).

⚠ DANGER — DESTRUCTIVE STAGE

Never delete or prune a backup archive, and never restore over a live VMID. The two archives of VM 297 (`ubuntu-vm`, destroyed on or before 2026-09-19) are the only remaining copy of that guest and must be kept.

15.1 Backup and restore matrix

Rendered from [docs/reference/backup-matrix.yml](#); edit that file, then run `make docs-reference`.

Item	What	Schedule	Destination	Restore	Verify	Status
proxmox-guests	Whole-guest vmdump archives for VMs 290, 300, 310, 320-322, 330	inventory/lab.yml backup_jobs (daily; VM 290 weekly Sunday 01:00)	synology-backup NFS, NAS Volume 1 (single disk — no redundancy)	qmrestore - storage - unique 1, every NIC link_down=1, then inspect	zstdcat vma verify - ; plus the archive's SHA-256 sidecar	backup-restored
proxmox-second-copy	Newest finalized vmdump archive per VMID, retained on workstation	Daily 06:30 while logged in; catch up after missed start; never wake the PC	D:/homelab-backups on workstation (second on-site copy)	Isolated qm-restore from the retained D copy; platform/proxmox/docs/backup-pull.md	Client size and SHA-256 match NAS list before and after; NAS_UNCHANGED=true	declared

Item	What	Schedule	Destination	Restore	Verify	Status
vm300-app-data	NPM, Linking, Speedtest Tracker SQLite; Uptime Kuma 2 MariaDB dump; Homepage/ NPM config and certificates	daily 02:45 (homelab-app-backup.timer)	/srv/homelab-backups/staging on VM 300, carried off-guest by the 03:15 VM 300 vxdump	On an isolated restore of VM 300: copy each SQLite file back under /srv/homelab/ with the app stopped; for Uptime Kuma, gunzip -c uptime-kuma-all-data-bases.sql.gz docker exec -i uptime-kuma mariadb -socket=/app/data/run/mariadb.sock	sqlite3 'PRAGMA integrity_check' returns ok; gunzip -t the Kuma dump; uptime_kuma_monitors.py -dump reports no drift	backup-restored
vm310-observability	Prometheus TSDB snapshot, grafana.db, configuration	daily 03:30 (homelab-observability-backup.timer)	/srv/observability/backups on VM 310, carried off-guest by the 04:15 VM 310 vxdump	make -C observability restore-drill ARCHIVE=/srv/observability/backups/ (never overwrites live data)	The restore drill's checks pass; homelab_observability_backup_last_success_timestamp_seconds is fresh	backup-restored
vm330-netbox	NetBox PostgreSQL (plain SQL, serializable-deferrable) and media/reports/scripts	daily 03:45 (netbox-app-backup.timer)	/srv/netbox-backups/staging on VM 330, carried off-guest by the 04:45 VM 330 vxdump	On an isolated VM 330 copy with an empty database: gunzip -c /netbox.sql.gz docker exec -i netbox-postgres psql -U netbox -d netbox	gunzip -t the dump; NetBox login works and object counts match the source	backup-restored
openbao-raft	Encrypted OpenBao Raft snapshots	configured only after initialization (make -C security/secrets-openbao configure-backup)	Synology, encrypted with an age identity kept outside OpenBao and the NAS	security/secrets-openbao/docs/restore-runbook.md in a protected VM on an isolated bridge	make -C security/secrets-openbao inspect-backup SNAPSHOT=AGE_IDENTITY=	manual
arr-apps	Radarr and Prowlarr scheduled backup zips (database and config)	daily 05:40 (arr-backup.timer); only while workstation is awake	NAS media-backups share (Volume 2) under arr/, with SHA-256 sidecars	On an isolated guest, Radarr/ Prowlarr System > Backup > Restore from the copied zip	sha256sum -c .sha256; python3 -m zipfile -t	backup-restored

Item	What	Schedule	Destination	Restore	Verify	Status
plex-metadata	Plex database backups and Preferences.xml (holds the Plex token)	daily 04:30 while the desktop user is logged on (plex role, gated)	NAS media-backups share under plex/ (operator-only ACL)	Stop Plex; copy the dated com.plex-app.plugins.library.db backup and Preferences.xml back; start Plex	python services/media/scripts/plex_libraries.py check, then plan shows no drift	backup-restored
media-library	Movies and Downloads on the NAS Media share	not applicable	none — protected by the RAID1 mirror only	Not recreated from backup: rebuild the mirror from the healthy disk after replacing bay 3 (platform/nas/docs/synology-storage-health.md)	DSM Storage Manager shows the pool healthy; no Btrfs snapshots exist, and Disk 3 has failed SMART	manual
qbittorrent-state	qBittorrent settings and torrent list on Windows	not applicable	none	Reinstall and set the Web UI port, credentials and M:path by hand (L0/L7)	Radarr's download-client test succeeds	manual
media-guest-disk	media-vm system and data disks	not applicable	none — rebuilt, not restored	Rebuild with L0 (cloud-init template) + L4/L5 (Ansible) and restore arr-apps above	make -C services/media preflight, then the L5 media verifications	declared
tofu-state	Encrypted OpenTofu state for every root	Operator copy after every verified state write; unique names retained forever	Encrypted timestamped copies on NAS proxmox-cluster-backups/opentofu-state/	Restore an encrypted copy to D:/homelab-state; otherwise re-adopt by import	SHA-256 matches both copies; strict encryption plan shows no changes	manual
nas-configuration	DSM configuration (shares, accounts, DNS zone, NFS exports)	after every DSM change	operator's encrypted storage (manual DSM configuration export)	DSM Control Panel > Update & Restore > Configuration Backup	Shares, accounts and DNS records in lab.yml exist after restore	manual

Source: <docs/reference/generated/backups.md>

16 Troubleshooting

Symptom	Likely cause	First response
A stage fails immediately with “Set ... to the exact string”	Gate not passed, or the confirmation differs by a character	Copy the expected string from Appendix C ; pass it in <code>EXTRA_JSON</code>
Approval set but destructive tasks skipped	<code>--tags all,destructive</code> missing	Use the Make target, which passes the tags
SSH fails with a changed host key	Cloud-init re-ran on a stopped guest, or a reinstall	Verify through <code>qm guest exec ... ssh-keygen -lf</code> ; the operator re-pins
Private names fail on a guest, public names work	NAS DNS down, or the routing-domain drop-in missing	Check <code>resolvectl status</code> ; never point the guest wholesale at the NAS
A freshness alert never fires	Its input series does not exist	Query the metric first; check <code>node_textfile_scrape_error</code>
Backup timer failed for days, no alert	Unit failure is not a metric	Check <code>journalctl -u <timer></code> ; the freshness metric covers it now
<code>make docs</code> fails with “snippet ... changed”	The source lines moved or changed	Review the new lines, fix the range, run <code>build.py --fill</code>
CI <code>inventory contract</code> fails	A live identity changed in one place only	Change <code>inventory/lab.yml</code> and the consumer together

The facts behind most of these rows were learned against the live hosts. They are recorded once, in the lessons-learned reference, included here:

16.1 Lessons learned against the live lab

Verified against the live hosts. Do not re-derive these. Moved verbatim from the root `CLAUDE.md` in Phase 2 (2026-09-29).

16.1.1 Environment facts learned the hard way

Verified against the live hosts. Do not re-derive these.

- **A metric can be written and still never exist.** `backup.sh` writes `homelab_observability_backup_last_success_timestamp_seconds` into `/var/lib/node_exporter/textfile/` as mode **0600 root:root**. The `node_exporter` container runs as **nobody** and cannot read it, so `node_textfile_scrape_error` = 1 and Prometheus has **no such series**. Discovered 2026-09-19. Consequence: any alert written against that metric is **vacuous** — it evaluates against nothing and can never fire, which looks identical to “healthy”. Whenever you add an alert, first confirm its input series actually returns data, and alert on `node_textfile_scrape_error > 0` so the collector cannot fail silently again.
- **A backup can fail for days while monitoring is perfectly healthy.** VM 310’s application backup failed six consecutive nights, 2026-09-11 → 09-16, with `curl: (7) Failed to connect to 192.168.0.31 port 9090`. Prometheus had **continuous data with no gap over 10 minutes** throughout, so the stack was not down and the “stack was down” explanation is wrong. **Root cause remains unresolved.** This is the second unexplained multi-day backup gap, after 2026-08-28 → 09-05. `systemctl --failed` is useless retrospectively: later successful runs clear the failed state.

- `qm set <vmid> --nameserver ...` on a **STOPPED** cloud-init guest regenerates the cloud-init drive, and on the next boot cloud-init re-runs and **REGENERATES THE GUEST'S SSH HOST KEYS**. This happened on 2026-09-06 to VMs 320 `bao-1` and 321 `bao-2`: their pinned `known_hosts` fingerprints were invalidated by a routine nameserver correction. Guests that were **running** when the same change was applied (322, 330) kept their keys, because cloud-init never re-ran. Before changing cloud-init settings on a stopped guest, expect a host key change and plan to re-verify the fingerprint out of band.
- **A guest's SSH host key can be verified out of band through the hypervisor** with `qm guest exec <vmid> -- /usr/bin/ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub`. This does not traverse the network SSH path, so it is a valid independent channel for confirming that a changed key is the guest's own key and not an interception. It is the same method used to verify VM 300 in Stage C3. It still does **not** authorise auto-accepting the key — the operator updates the pin.
- `pvesm status|list --output-format json` **does not exist** on PVE 8.2 or 9.1. Use `pvesh get /nodes/<node>/storage ...` instead.
- `pvesm status` **column 6 is Available**, not `$5-$6`.
- `/etc/pve/qemu-server, lxc and local` **are symlinks** into `/etc/pve/nodes/<node>/`. `tar` needs `-h` or the guest configs come out empty while the play still reports success.
- **A regex backreference inside a folded (`>-`) or literal (`|-`) YAML scalar is passed through verbatim** by Ansible — `regex_search(..., '\1')` yields the string `\1`. Plain and double-quoted scalars are fine. This silently broke the `pve8to9` failure counter, which then evaluated to 0 and would have approved a failed upgrade. A test now guards it.
- `mount.nfs` **rejects `vers=` and `nfsvers=` together**. The version is negotiated by the probe mount, not hardcoded.
- `ansible.builtin.unarchive` **requires GNU tar**, absent on a stock macOS controller. Use `tar -xzf` directly.
- `community.general.yaml` **stdout callback was removed** in v12; use the built-in default callback with `result_format = yaml`.
- **Proxmox CLI tools have no `--help`**. Use `pvesh usage <api-path> -v`.
- **Ansible's `never` tag**: `--tags all,destructive` runs everything including never-tagged destructive tasks. Verified.
- The backup role must create `/root/.pve-migration/manifests` before the first archive. The first live Stage 3 run exposed this omission after writing and hashing `EVENG`; the archive was kept and the role now creates the directory.
- Bash `PIPESTATUS` cannot be read in the parent after wrapping a pipeline in command substitution. The first archive-verification run exposed this. The verifier now runs the pipeline in the current shell and captures both statuses.
- `gathering=smart` can reuse a cached `ansible_date_time` in evidence. Gathering is now explicit, and backup timestamps come from the archive file's own `mtime`.
- Ansible's key/value `-e` parser splits multi-word values even when the shell delivered them as one argument. Approval examples now use `EXTRA_JSON` so exact confirmation phrases are passed as structured JSON.
- Role defaults are scoped to the play that invokes the role. The first full Stage 4b run verified all four archives but its localhost adjudication could not see `backup_verify_generations`; the variable now lives in `group_vars/all`.

16.1.2 Working style that has paid off here

Measure before asserting. Two plausible root-cause hypotheses (snapshots, then disk failure) were both wrong and were only caught by running commands against the live systems. Prefer a read-only command over an inference, and say plainly when a previous claim turns out to be wrong.

`du` on the NAS is close to useless — ARM CPU, millions of files, scans run for hours. Prefer DSM's own reporting, `bttrfs` tooling, or per-child parallel scans.

Source: [docs/reference/lessons-learned.md](#)

17 Glossary

Term	Meaning
Allow flag	A boolean such as <code>netbox_allow_deploy</code> ; defaults <code>false</code>
Confirmation	An exact string, often embedding the host, that must match before a stage runs
EXTRA_JSON	The Make variable that carries approvals as JSON to <code>ansible-playbook -e</code>
Import-first	An OpenTofu root that adopts existing guests with <code>import</code> blocks before managing them
Rebuild-only root	A create-mode OpenTofu root used only after total loss (ADR-0007)
Rung	One level (L0–L8) of the rebuild ladder
Routing domain	A systemd-resolved <code>~domain</code> that sends only that zone to a given server
<code>keep-all=1</code>	Proxmox prune policy that retains every archive
Isolated restore	A restore into a spare protected VMID with every NIC link-down
vzdump	Proxmox's whole-guest backup; archives are <code>.vma.zst</code>
VIP	The Keepalived virtual address (192.168.0.40) in front of OpenBao
Ratchet	A list of accepted exceptions that tests only allow to shrink
Split horizon	The lab zone resolves only inside the LAN, from the NAS

18 Appendix A: Make targets

Generated from every Makefile by [tools/docs/generate_reference.py](#). Targets without help text print `—`.

18.1 Make targets

18.1.1 repository root

Target	What it does
help	—
rebuild-plan	—
docs-check	—
docs-reference	—
docs-diagrams	—
docs	—

18.1.2 observability

Target	What it does
help	List independently gated stages
setup	Install local controller tooling
install-hooks	Install pinned pre-commit secret and safety checks
lint	Validate YAML, Ansible, Compose, Prometheus, and secret hygiene
test	Run offline repository tests
syntax	Syntax-check every playbook without contacting a host
check	Run all offline validation
preflight	Read-only discovery; writes artifacts/preflight.json on controller
bootstrap	Harden the already-created observability VM; no stack deployment
deploy	Deploy central stack only after reviewing preflight and providing secrets
validate	Read-only runtime validation
update-dashboards	Gated dashboard/rule update without restarting containers
backup	Trigger one application-aware backup on the observability VM
restore-drill	Validate a named backup archive without overwriting live data

18.1.3 platform/nas

Target	What it does
help	—
setup	—
lint	—
test	—
syntax	—
check	—
plan	—
preflight	—
deploy	—

18.1.4 platform/proxmox

Target	What it does
help	Show this help
setup	Create the local virtualenv and install controller dependencies
lint	Run yamllint + ansible-lint + ruff + shellcheck (offline)
test	Run the offline unit test-suite
syntax	Syntax-check every migration playbook without contacting a host
check	Run all offline validation
tree	Print the repository tree
clean-artifacts	Remove generated reports (NEVER touches backups on the NAS)
configure-backup-pull	Gated: authorize the restricted workstation pull key on ONE node
validate-ssh	Stage 0a: prove existing SSH key auth works on all three hosts
discover	Stage 0b: read-only discovery of all three hosts
nas-preflight	Stage 1: validate the Synology NFS target from all three hosts
capture-config	Stage 2: capture sanitized host configuration from .11 and .12
backup-initial	Stage 3: snapshot-mode backup (usage: make backup-initial HOST=pve1)
backup-final	Stage 4: stopped-guest backup (usage: make backup-final HOST=pve1)
verify-backups-initial	Stage 3b: verify the initial generation before final shutdown
verify-backups	Stage 4b: checksum + vma/tar verify every archive

Target	What it does
prepare-pve3	Stage 5a: emit the .13 clean-install checklist / validate after reinstall
test-restore	Legacy plan only (not used by the current rebuild-later migration)
precheck-upgrade-pve1	Stage 6a: pve8to9 -full + repository change report for .11
upgrade-pve1	Stage 6b: gated PVE 8 -> 9 upgrade of .11 (requires console access)
create-cluster	Stage 7: create the cluster on .11 only
join-pve3	Stage 8: join .13 (requires allow_join_pve3 + confirmation string)
precheck-destroy-pve2	Stage 9a: verify every gate before .12 may be clean-installed
postinstall-pve2	Stage 9b: validate .12 after its manual clean install
join-pve2	Stage 9c: join .12 to the cluster
build-restore-plan	Legacy plan only: generate the retired .12 restore plan
restore-pve2	Legacy plan only: restore .12 guests (not in current migration)
start-restored-guests	Legacy plan only: start restored .12 guests
lan-scan	Read-only LAN sweep -> artifacts/lan-inventory.json
lan-site	Build the static LAN inventory page -> site/index.html
lan-preview	Serve site/ locally at http://127.0.0.1:8899
lan-deploy	Copy the page to the Synology (requires Web Station + a Volume 1 docroot)
homelab-preflight	Read-only validation of VM 300 before reconciliation
homelab-base	Reconcile VM 300 Docker, hardening, and filesystem prerequisites
homelab-service	Reconcile one app (usage: make homelab-service SERVICE=homepage)
homelab-backups	Reconcile VM 300 application-aware backup timer
netbox-preflight	Read-only validation of the NetBox guest before reconciliation
netbox-base	Reconcile NetBox guest Docker, hardening, and prerequisites
netbox-deploy	Deploy NetBox (requires the documented approval, see docs/netbox-plan.md)
netbox-backups	Reconcile the NetBox application-aware backup timer
netbox-terraform-fmt	Check planned NetBox guest Terraform formatting
netbox-terraform-init	Initialize the pinned Proxmox provider without a backend
netbox-terraform-validate	Validate the planned NetBox guest Terraform configuration
netbox-terraform-plan	Plan the planned NetBox guest. Apply stays a manual, reviewed step.

Target	What it does
terraform-fmt	Check Home Lab guest Terraform formatting
terraform-init	Initialize the pinned Proxmox provider without a backend
terraform-validate	Validate Home Lab guest Terraform configuration
terraform-plan	Import-first, non-destructive Terraform plan for VMs 300 and 310
rebuild-l0-answer-files	L0: render Proxmox answer files (dry run unless WRITE=1)
rebuild-l1-host-config	L1: vzdump jobs, onboot, startup from lab.yml (gated by EXTRA_JSON)
rebuild-guests-fmt	Check rebuild-only root formatting
rebuild-guests-validate	Validate the rebuild-only root (init -backend=false)
rebuild-guests-test	Offline tofu test with a mocked provider
rebuild-guests-plan	L3 operator plan; needs REBUILD_VARS='-var-file=...' with names + confirmation
validate	Stage 11: full three-node cluster validation + migration report

18.1.5 security/aaa-freeradius

Target	What it does
help	—
setup	—
lint	—
test	—
syntax	—
check	—
plan	—
preflight	—
deploy	—

18.1.6 security/secrets-openbao

Target	What it does
help	Show independently gated targets
setup	Install pinned offline-controller dependencies
lint	Run repository, Python, YAML, and Ansible safety checks
test	Run offline unit tests
syntax	Syntax-check every playbook without contacting a host
check	Run all offline validation
terraform-fmt	Check Terraform formatting
terraform-init	Initialize the pinned provider without a backend

Target	What it does
terraform-validate	Validate Terraform without contacting Proxmox
terraform-plan	Produce a reviewed plan; no apply target exists
discover-plan	Refresh sanitized secret-name and Git-history inventories
preflight	Read-only guest validation after first boot
prepare-hosts	Gated Ubuntu hardening and firewall stage
install-server	Gated signed OpenBao binary and systemd installation
configure-tls	Gated operator-supplied TLS deployment
configure-ha	Gated HAProxy/Keepalived and OpenBao configuration
validate-uninitialized	Prove TLS works and OpenBao remains uninitialized
configure-post-init	Gated policies/auth/engines after the attended ceremony
validate-post-init	Read-only proof of quorum, mounts, auth, audit, and TLS
configure-backup	Gated encrypted Raft snapshot timer installation
configure-monitoring	Gated OpenBao telemetry and audit shipping integration
backup-now	Trigger one encrypted snapshot on the elected backup node
inspect-backup	Offline decrypt/inspect of one snapshot (SNAPSHOT and AGE_IDENTITY required)
recovery-bundle	Build a secret-free configuration recovery bundle

18.1.7 `services/homelab-services`

Target	What it does
help	—
setup	—
lint	—
test	—
syntax	—
check	—
plan	—
preflight	—
deploy	—
backup	—

18.1.8 `services/media`

Target	What it does
help	List the media stages
setup	Create the venv and install pinned tooling and collections
lint	yamllint, ansible-lint, ruff

Target	What it does
test	Offline pytest
syntax	Syntax-check every playbook without contacting a host
check	Run all offline validation
plan	Offline: validate and test the *arr root with mocked providers
preflight	Read-only media guest preflight
configure-host	Gated: packages, directories, NAS automounts
deploy-arr	Gated: deploy the digest-pinned *arr Compose project
deploy	Alias of deploy-arr (one stage)
sync-quality-profiles	Recyclarr -preview, or a real sync behind its own gate
check-plex	Read-only Plex check; backup task behind its gate
configure-backup	Gated: nightly append-only *arr backup copy
arr-fmt	Check *arr root formatting
arr-validate	Validate the *arr root (init -backend=false)
arr-test	Offline tofu test with mocked providers
arr-plan	Operator plan from workstation; needs backend.hcl and TF_VAR_* inputs

18.1.9 services/netbox

Target	What it does
help	—
setup	—
lint	—
test	—
syntax	—
check	—
plan	—
preflight	—
deploy	—
backup	—

18.1.10 tests

Target	What it does
help	—
setup	—
lint	—
test	—

Target	What it does
syntax	—
check	—

Source: [docs/reference/generated/make-targets.md](#)

19 Appendix B: Workflows

19.1 Workflows

Workflow	Triggers	Job	Runner	Environment
<code>_component-checks.yml</code>	workflow_call	checks	ubuntu-24.04	—
<code>ci.yml</code>	push, pull_request	changes	ubuntu-24.04	—
<code>ci.yml</code>	push, pull_request	component	reusable	—
<code>ci.yml</code>	push, pull_request	contracts	ubuntu-24.04	—
<code>ci.yml</code>	push, pull_request	secrets	ubuntu-24.04	—
<code>ci.yml</code>	push, pull_request	docs	ubuntu-24.04	—
<code>ci.yml</code>	push, pull_request	music-library	ubuntu-24.04	—

Source: [docs/reference/generated/workflows.md](https://docs.reference/generated/workflows.md)

20 Appendix C: Approval gates

20.1 Approval gates

Every gate defaults the flag to `false` and the confirmation to `""`. Expressions using `inventory_hostname` are filled in with the target host.

Flag	Confirmation variable	Expected confirmation	Asserted in
<code>observability_allow_bootstrap</code>	<code>observability_bootstrap_confirmation</code>	('BOOTSTRAP' ~ inventory_hostname)	<code>observability/ansible/playbooks/20-bootstrap.yml</code>
<code>observability_allow_deploy</code>	<code>observability_deploy_confirmation</code>	('DEPLOY OBSERVABILITY STACK ON' ~ inventory_hostname)	<code>observability/ansible/playbooks/40-deploy.yml</code>
<code>allow_observability_dashboard_update</code>	<code>observability_dashboard_update_confirmation</code>	UPDATE HOME LAB OBSERVABILITY DASHBOARDS	<code>observability/ansible/playbooks/41-update-dashboards.yml</code>
<code>allow_configure_nfs_storage</code>	–	flag only	<code>platform/nas/ansible/roles/synology_nfs/tasks/configure.yml</code>
<code>allow_repo_change_pve1</code>	<code>repo_change_confirmation_pve1</code>	CHANGE REPOSITORIES pve1 192.168.0.11	<code>platform/proxmox/ansible/playbooks/migration/41-upgrade-pve1.yml</code>
<code>allow_upgrade_pve1</code>	<code>upgrade_confirmation_pve1</code>	UPGRADE pve1 192.168.0.11 TO PVE 9	<code>platform/proxmox/ansible/playbooks/migration/41-upgrade-pve1.yml</code>
<code>allow_destroy_pve2</code>	<code>destroy_confirmation_pve2</code>	DESTROY pve2 192.168.0.12 AND DISCARD ALL FIVE GUESTS	<code>platform/proxmox/ansible/playbooks/migration/71-postinstall-pve2.yml</code>
<code>allow_restore_pve2</code>	<code>restore_confirmation_pve2</code>	RESTORE pve2 GUESTS FROM VERIFIED BACKUP	<code>platform/proxmox/ansible/playbooks/migration/80-restore-pve2.yml</code>
<code>allow_start_restored_guests</code>	<code>start_restored_confirmation</code>	START RESTORED GUESTS ON pve2	<code>platform/proxmox/ansible/playbooks/migration/81-start-restored-guests.yml</code>
<code>allow_backup_pull</code>	<code>backup_pull_confirmation</code>	'INSTALL BACKUP PULL ON' ~ inventory_hostname ~ ' ~ ansible_host	<code>platform/proxmox/ansible/roles/backup_pull/tasks/main.yml</code>
<code>allow_create_cluster</code>	<code>create_cluster_confirmation</code>	CREATE CLUSTER homelab ON pve1 192.168.0.11	<code>platform/proxmox/ansible/roles/pve_cluster/tasks/main.yml</code>
<code>pve_host_config_allow</code>	<code>pve_host_config_confirmation</code>	L1 CONFIGURE {{ ansible_play_hosts_all sort join(' ') }}	<code>platform/proxmox/ansible/roles/pve_host_config/tasks/main.yml</code>

Flag	Confirmation variable	Expected confirmation	Asserted in
pve_join_allow	pve_join_confirmation	set per stage by the calling playbook (pve_join_confirmation_required)	platform/proxmox/ansible/roles/pve_join/tasks/main.yml
allow_guest_shutdown	guest_shutdown_confirmation	SHUTDOWN GUESTS FOR FINAL BACKUP	platform/proxmox/ansible/roles/vz-dump_backup/tasks/shutdown_guests.yml
allow_prepare_openbao_hosts	prepare_openbao_hosts_confirmation	PREPARE OPENBAO HOSTS 320 321 322	security/secrets-openbao/ansible/playbooks/10-prepare-hosts.yml
allow_install_openbao	install_openbao_confirmation	INSTALL OPENBAO 2.6.1 WITHOUT INITIALIZING	security/secrets-openbao/ansible/playbooks/20-install-server.yml
allow_configure_openbao_tls	configure_openbao_tls_confirmation	DEPLOY OPENBAO TLS TO THREE NODES	security/secrets-openbao/ansible/playbooks/30-configure-tls.yml
allow_configure_openbao_ha	configure_openbao_ha_confirmation	CONFIGURE OPENBAO RAFT AND VIP UNINITIALIZED	security/secrets-openbao/ansible/playbooks/40-configure-ha.yml
allow_configure_openbao_post_init	configure_openbao_post_init_confirmation	CONFIGURE OPENBAO POST INIT NO SECRET VALUES	security/secrets-openbao/ansible/playbooks/60-configure-post-init.yml
allow_configure_openbao_backup	configure_openbao_backup_confirmation	CONFIGURE ENCRYPTED OPENBAO RAFT BACKUPS	security/secrets-openbao/ansible/playbooks/70-configure-backup.yml
allow_configure_openbao_monitoring	configure_openbao_monitoring_confirmation	CONFIGURE OPENBAO MONITORING AND AUDIT SHIPPING	security/secrets-openbao/ansible/playbooks/80-configure-monitoring.yml
allow_run_openbao_backup	run_openbao_backup_confirmation	RUN ONE OPENBAO RAFT BACKUP	security/secrets-openbao/ansible/playbooks/90-backup-now.yml
homelab_allow_guest_config	homelab_guest_config_confirmation	('CONFIGURE GUEST' ~ inventory_hostname)	services/homelab-services/ansible/playbooks/30-configure-guest.yml
homelab_allow_service_deploy	homelab_service_deploy_confirmation	('DEPLOY' ~ homelab_services_service_name ~ ' ON ' ~ inventory_hostname)	services/homelab-services/ansible/playbooks/40-deploy-service.yml
homelab_allow_backup_config	homelab_backup_config_confirmation	('CONFIGURE BACKUPS ON' ~ inventory_hostname)	services/homelab-services/ansible/playbooks/50-configure-backups.yml
homelab_services_allow_npm_route_ownership	homelab_services_npm_route_confirmation	REPLACE NPM UI PROXY HOSTS WITH DECLARATIVE HOME LAB ROUTES	services/homelab-services/ansible/roles/homelab_services/tasks/main.yml
arr_stack_allow	arr_stack_confirmation	('MEDIA DEPLOY' ~ inventory_hostname)	services/media/ansible/roles/arr_stack/tasks/main.yml

Flag	Confirmation variable	Expected confirmation	Asserted in
media_backup_allow	media_backup_confirmation	('MEDIA BACKUP' ~ inventory_hostname)	services/media/ansible/roles/media_backup/tasks/main.yml
media_host_allow	media_host_confirmation	('MEDIA HOST' ~ inventory_hostname)	services/media/ansible/roles/media_host/tasks/main.yml
–	plex_backup_confirmation	('PLEX BACKUP' ~ inventory_hostname)	services/media/ansible/roles/plex/tasks/main.yml
net-box_allow_guest_config	net-box_guest_config_confirmation	('CONFIGURE GUEST' ~ inventory_hostname)	services/netbox/ansible/playbooks/31-configure-guest.yml
netbox_allow_deploy	net-box_deploy_confirmation	('DEPLOY NETBOX ON' ~ inventory_hostname)	services/netbox/ansible/playbooks/40-deploy.yml
net-box_allow_backup_config	net-box_backup_config_confirmation	('CONFIGURE BACKUPS ON' ~ inventory_hostname)	services/netbox/ansible/playbooks/50-configure-backups.yml
net-box_app_allow_first_deployment	net-box_app_first_deployment_confirmation	DEPLOY NETBOX ON VM 330	services/netbox/ansible/roles/netbox_app/tasks/main.yml

Source: [docs/reference/generated/gates.md](#)

21 Appendix D: Operator inputs

21.1 Operator inputs (L7)

Every secret or attended input the lab depends on: what it is, where it lives, and when it is used. **No value is ever stored in Git, printed, or handled by an agent.** Installed files are root-owned mode 0600 and checked by metadata only. OpenBao initialization and unsealing stay attended.

Input	What it is	Where it lives	Used at
Proxmox root password hash	crypt(3) hash for answer files	Operator password manager; PVE_ROOT_PASSWORD_HASH at build time	L0
Root SSH public keys	Keys baked into answer files	PVE_ROOT_SSH_KEYS_FILE (public keys only)	L0
Notification address	mailto for answer files	PVE_MAILTO at build time	L0
Node install inputs	FQDN, disk and NIC filters	inventory/lab.yml nodes.*.install (not secret)	L0
DSM admin credentials	NAS administration	Operator password manager; never in automation	L0, L1
Cluster SSH key	~/.ssh/proxmox_cluster_ed25519	Operator controller only	L1-L3, L8
Root password prompt for pvecm add	Join authentication	Typed by the operator at the prompt	L2
Proxmox API token	Dedicated token for OpenTofu	TF_VAR_proxmox_api_token in the operator shell	L3
State passphrase	Per-root PBKDF2/AES-GCM state and plan encryption	Operator password manager; TF_VAR_state_passphrase in local devcontainer only	L3, L5 (*arr)
State locations	D1 local backend and encrypted NAS copies	D:\homelab-state\<root>\; NAS proxmox-cluster-backups/opentofu-state/<root>/	L3
Guest admin SSH keys	Per-guest keys (homelab_services_ed25519, homelab_observability_ed25519, homelab_netbox_ed25519, OpenBao key)	Operator controller; public halves in rebuild.tfvars (git-ignored)	L3-L5
Guest host-key pins	Fingerprints verified out of band (guest agent)	known_hosts; recorded in lab.yml	L4
VM 300 app secrets	NetKnife, Speedtest SMTP, DNS-01 credential, ...	/etc/homelab/secret-store/ on VM 300	L5
Observability secrets	PVE exporter token, Grafana admin, SNMPv3, SMTP	/etc/observability/secret-store/ on VM 310	L5
NetBox superuser and API token	NetBox administration	Operator-installed on VM 330	L5

Input	What it is	Where it lives	Used at
OpenBao TLS material	Node certificates from the offline root	Operator-installed on VMs 320-322	L5
Offline root CA key	Signs OpenBao node certificates	<code>~/.local-credentials/home-lab/openbao/pki/</code> → move to encrypted custody	L5
OpenBao unseal shares	Five PGP-encrypted shares, threshold three	D4: sole operator, five off-line keys/places; only public keys/fingerprints near Git or agents; never on NAS	L7, attended 2026-10-10; verify VM 320-322 archives the day before
OpenBao backup age identity	Decrypts Raft snapshots	Outside OpenBao and the NAS	L6
NAS CIFS credential (media)	<code>plex-reader/media</code> account for the guest mounts	<code>/etc/media/secrets/cifs.cred</code> on media-vm	L4
Radarr / Prowlarr API keys	App API keys	<code>/etc/media/secrets/*.env</code> on the guest; <code>TF_VAR_radarr_api_key</code> , <code>TF_VAR_prowlarr_api_key</code> for the *arr root	L5
qBittorrent Web UI login	Download client credentials	<code>TF_VAR_qbittorrent_username</code> , <code>TF_VAR_qbittorrent_password</code> ; qBittorrent on Windows	L5
Plex claim	One-time claim in the operator's browser	Never stored	L5 (attended)
Plex token	<code>PLEX_TOKEN</code> for the library script	Operator password manager; also inside <code>Preferences.xml</code> backups	L5, L6
Media guest connection	SSH forward port and admin user	<code>inventory/host_vars/media-vm.yml</code> (git-ignored)	L4-L6
Backup pull private key	Dedicated passphrase-free ED25519 key; operator generates it	<code>~/.ssh/homelab_backup_pull_ed25519</code> on workstation only; agent handles only <code>.pub</code>	L6
Backup node public key	Public ED25519 host key, compared with <code>lab.yml</code> fingerprint before pinning	<code>D:\homelab-backups\control\known_hosts</code> after verification	L6
Rotation list	Credentials that must be rotated	<code>MEMORY.md</code> → Rotation list	L7

Source: [docs/reference/operator-inputs.md](https://docs.reference/operator-inputs.md)

22 Appendix E: Versions and counts

22.1 Version matrix

Kind	Name	Pinned	Source
Container image	corentinh/it-tools:2024.10.22-7ca5933	sha256:6f177c156b94...	services/homelab-services/config/templates/stage-d/compose/it-tools/compose.yaml
Container image	ghcr.io/gethomepage/homepage:v1.13.2	sha256:c881120b024d...	services/homelab-services/config/templates/stage-d/compose/homepage/compose.yaml
Container image	ghcr.io/google/cadvisor:v0.60.5	(no digest)	observability/docker/compose.yaml
Container image	ghcr.io/onedr0p/expo-tarr:v2.3.0	sha256:af535d94061c...	services/media/config/compose.yaml
Container image	ghcr.io/onedr0p/expo-tarr:v2.3.0	sha256:af535d94061c...	services/media/config/compose.yaml
Container image	ghcr.io/recyclarr/recyclarr:8	sha256:6e69e009e1cd...	services/media/config/compose.yaml
Container image	ghcr.io/schlagmichdoch/pairdrop:v1.11.2	sha256:39fdf4aa90c5...	services/homelab-services/config/templates/stage-d/compose/pairdrop/compose.yaml
Container image	grafana/alloy:v1.18.0	(no digest)	observability/docker/compose.yaml
Container image	grafana/alloy:v1.18.0	sha256:491b0578c049...	services/media/config/compose.yaml
Container image	grafana/grafana:13.1.0	(no digest)	observability/docker/compose.yaml
Container image	grafana/loki:3.7.4	(no digest)	observability/docker/compose.yaml
Container image	homelab/netknife-api:28ef127baeab	(no digest)	services/homelab-services/config/templates/stage-d/compose/netknife/compose.yaml
Container image	homelab/netknife-web:28ef127baeab	(no digest)	services/homelab-services/config/templates/stage-d/compose/netknife/compose.yaml
Container image	jc21/nginx-proxy-manager:2.15.1	sha256:99a885f56ca2...	services/homelab-services/config/templates/stage-d/compose/npm/compose.yaml
Container image	louislam/uptime-kuma:2.3.2	sha256:dbede95c5296...	services/homelab-services/config/templates/stage-d/compose/uptime-kuma/compose.yaml

Kind	Name	Pinned	Source
Container image	lscr.io/linuxserver/prowlarr:latest	sha256:f2b26429893d...	services/media/config/compose.yaml
Container image	lscr.io/linuxserver/radarr:latest	sha256:adb6c09d6b72...	services/media/config/compose.yaml
Container image	lscr.io/linuxserver/speedtest-tracker:1.14.3	sha256:cdb782713a9c...	services/homelab-services/config/templates/stage-d/compose/speedtest-tracker/compose.yaml
Container image	prom/alertmanager:v0.32.1	(no digest)	observability/docker/compose.yaml
Container image	prom/blackbox-exporter:v0.28.0	(no digest)	observability/docker/compose.yaml
Container image	prom/node-exporter:v1.12.1	(no digest)	observability/docker/compose.yaml
Container image	prom/node-exporter:v1.12.1	sha256:1b4e4438faca...	services/media/config/compose.yaml
Container image	prom/prometheus:v3.12.0	(no digest)	observability/docker/compose.yaml
Container image	prom/snmp-exporter:v0.30.1	(no digest)	observability/docker/compose.yaml
Container image	prompve/prometheus-pve-exporter:3.9.0	(no digest)	observability/docker/compose.yaml
Container image	sissbruecker/linkding:1.45.0	sha256:5bc84e076626...	services/homelab-services/config/templates/stage-d/compose/linkding/compose.yaml
Galaxy collection	ansible.posix	>=1.5.4	platform/proxmox/requirements.yml
Galaxy collection	ansible.posix	2.2.2	services/media/requirements.yml
Galaxy collection	ansible.utils	>=5.1.0	platform/proxmox/requirements.yml
Galaxy collection	ansible.windows	3.8.0	services/media/requirements.yml
Galaxy collection	community.docker	5.0.6	observability/requirements.yml
Galaxy collection	community.docker	5.0.6	services/media/requirements.yml
Galaxy collection	community.general	12.4.0	observability/requirements.yml
Galaxy collection	community.general	>=9.0.0	platform/proxmox/requirements.yml
Galaxy collection	community.general	12.4.0	services/media/requirements.yml
OpenTofu provider	bpg/proxmox	0.107.0	platform/proxmox/terraform/homelab-guests/versions.tf

Kind	Name	Pinned	Source
OpenTofu provider	bpg/proxmox	0.107.0	platform/proxmox/terraform/rebuild-guests/versions.tf
OpenTofu provider	bpg/proxmox	0.107.0	security/secrets-openbao/terraform/versions.tf
OpenTofu provider	bpg/proxmox	0.107.0	services/netbox/terraform/proxmox-guest/versions.tf
OpenTofu provider	devopsarr/prowlarr	3.2.1	services/media/terraform/arr/versions.tf
OpenTofu provider	devopsarr/radarr	2.5.0	services/media/terraform/arr/versions.tf
Python package	ansible-core	==2.19.4	observability/requirements.txt
Python package	ansible-core	>=2.16	platform/proxmox/requirements.txt
Python package	ansible-core	==2.19.2	security/secrets-openbao/requirements.txt
Python package	ansible-core	==2.19.4	services/media/requirements.txt
Python package	ansible-lint	==25.12.2	observability/requirements.txt
Python package	ansible-lint	>=24.2.0	platform/proxmox/requirements.txt
Python package	ansible-lint	==25.8.2	security/secrets-openbao/requirements.txt
Python package	ansible-lint	==25.12.2	services/media/requirements.txt
Python package	jmespath	>=1.0.1	platform/proxmox/requirements.txt
Python package	netaddr	>=1.2.1	platform/proxmox/requirements.txt
Python package	plexapi	==4.18.3	services/media/requirements.txt
Python package	pre-commit	==4.3.0	observability/requirements.txt
Python package	pytest	>=8.0.0	platform/proxmox/requirements.txt
Python package	pytest	==8.4.2	services/media/requirements.txt
Python package	pytest	==8.4.2	tests/requirements.txt
Python package	PyYAML	==6.0.3	services/media/requirements.txt
Python package	PyYAML	==6.0.3	tests/requirements.txt
Python package	ruff	>=0.5.0	platform/proxmox/requirements.txt

Kind	Name	Pinned	Source
Python package	ruff	==0.12.11	security/secrets-openbao/requirements.txt
Python package	ruff	==0.12.11	services/media/requirements.txt
Python package	ruff	==0.12.11	tests/requirements.txt
Python package	yamllint	==1.37.1	observability/requirements.txt
Python package	yamllint	>=1.35.1	platform/proxmox/requirements.txt
Python package	yamllint	==1.37.1	security/secrets-openbao/requirements.txt
Python package	yamllint	==1.37.1	services/media/requirements.txt
devcontainer	ansible-core	2.19.4	.devcontainer/Dockerfile
devcontainer	ansible-lint	25.12.2	.devcontainer/Dockerfile
devcontainer	gitleaks	8.30.1	.devcontainer/Dockerfile
devcontainer	mermaid-cli	12.0.0	.devcontainer/Dockerfile
devcontainer	node	24.21.0	.devcontainer/Dockerfile
devcontainer	opentofu	1.12.6	.devcontainer/Dockerfile
devcontainer	pandoc	3.12	.devcontainer/Dockerfile
devcontainer	pre-commit	4.3.0	.devcontainer/Dockerfile
devcontainer	pytest	8.4.2	.devcontainer/Dockerfile
devcontainer	ruff	0.12.11	.devcontainer/Dockerfile
devcontainer	weasyprint	70.0	.devcontainer/Dockerfile
devcontainer	yamllint	1.37.1	.devcontainer/Dockerfile

Source: <docs/reference/generated/versions.md>

22.2 Repository counts

Item	Count
Components with a Makefile	9
Ansible playbooks	56
Ansible roles	42
OpenTofu roots	6
Python test functions (before parametrisation)	290
OpenTofu test runs (mocked providers)	5
GitHub Actions workflows	2
Architecture decision records	9
Agent skills	10

Item	Count
Walkthrough diagrams	17
Proxmox nodes (lab.yml)	3
Guests (lab.yml)	8
Scheduled vzdump jobs (lab.yml)	7

Source: <docs/reference/generated/counts.md>